ETH zürich

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

CachePool: A scalable cache-coherent manycore architecture with private data cache

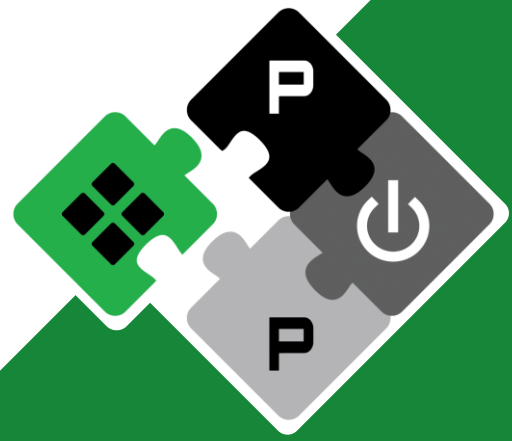
Integrated Systems Laboratory (ETH Zürich)

Master Thesis

Ho Tin Hung **hohung@student.ethz.ch**

Advisors: **Zexin Fu, Diyou Shen**

Supervisor: **Professor Dr. Luca Benini**



@pulp_platform

pulp-platform.org

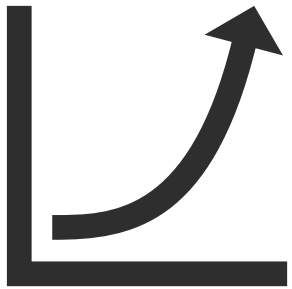
youtube.com/pulp_platform

PULP Platform

Open Source Hardware, the way it should be!

Rising Demand

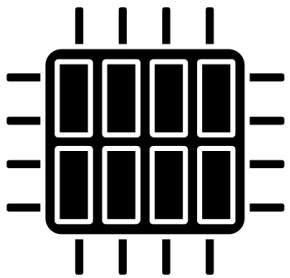
AI ML
6G

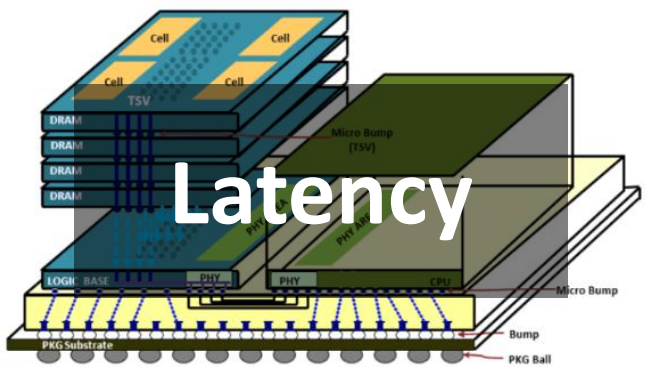


Intense computation
Large memory footprint

Hardware acceleration
Architectural innovation

Increase memory density
Quantization





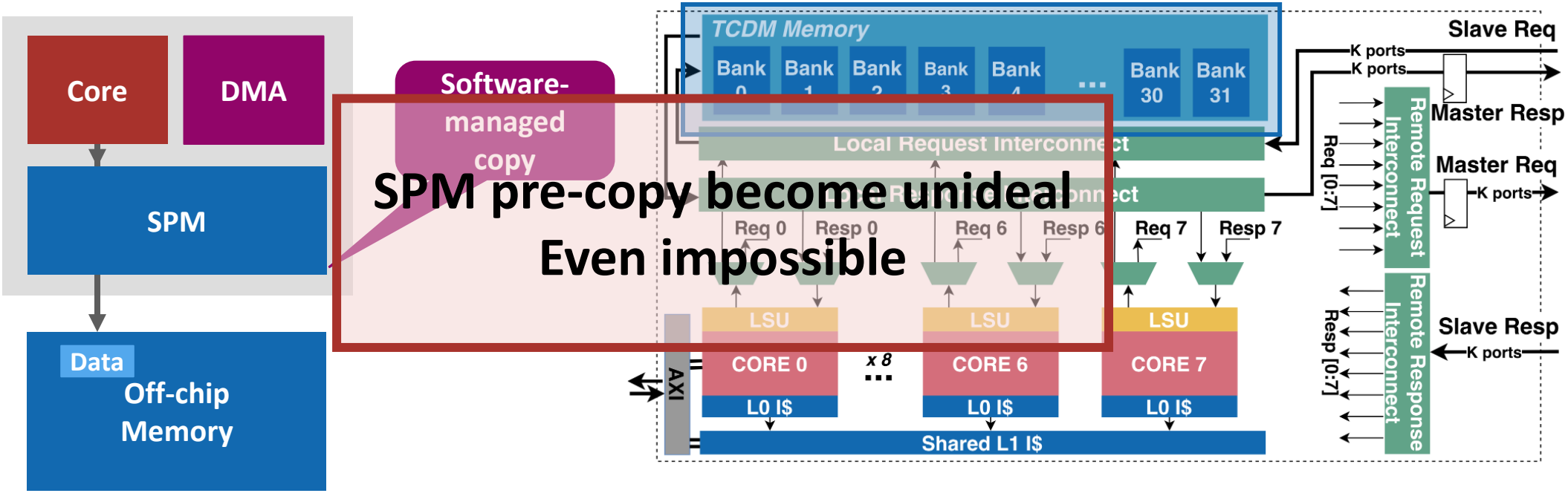
Latency

Sparsity

 Image Sources: https://www.top500.org/news/next-generation-3d-memory-in-the-works/; Gemini

SPM-Based Architecture

SoA solution: Scratchpad Memory



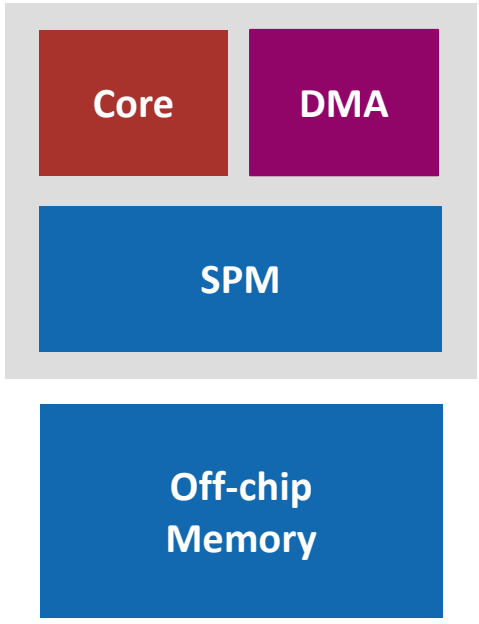
Lower hardware complexity
Energy and area efficiency

Large memory footprint
Sparsity
Indirect access

→

Programming difficulty
Bandwidth overhead
Unable to locate

Cache-based Memory



SW-managed

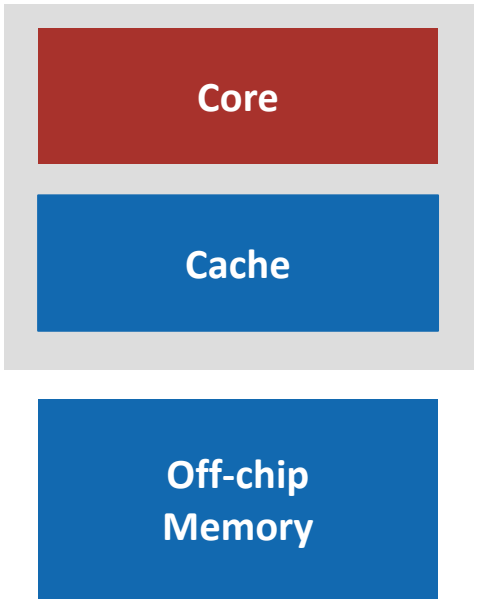
Manual copy

Pre-copy

Programming Difficulty

Unpredictability

Cache-based Memory



SW-managed → HW-managed

Manual copy → Direct Addressing

Pre-copy → Runtime Access

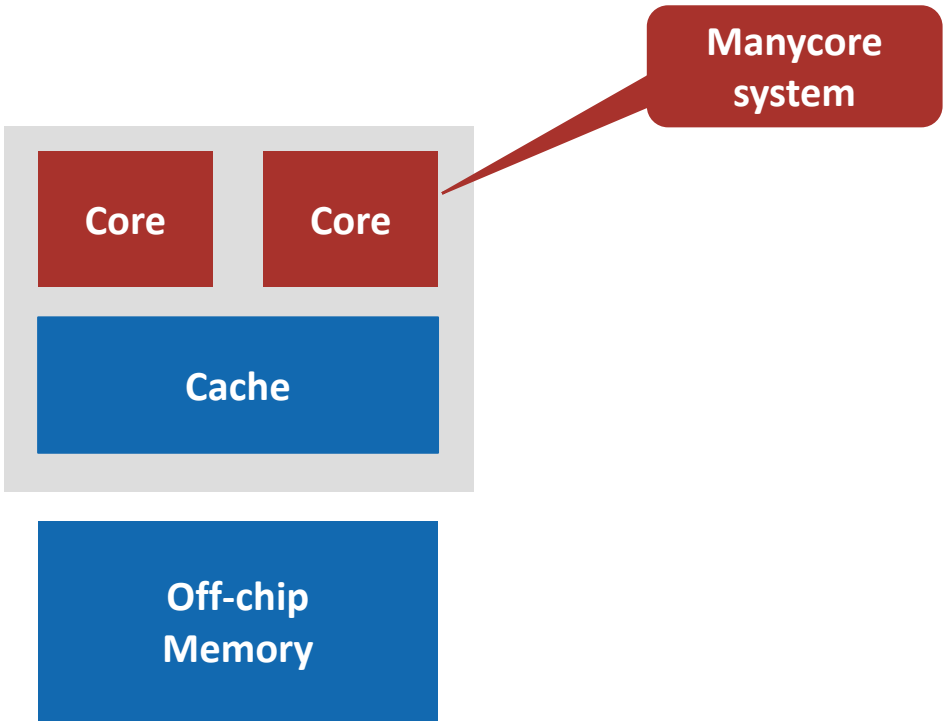
Programming Difficulty

Automatic data movement
Transparent to programmer

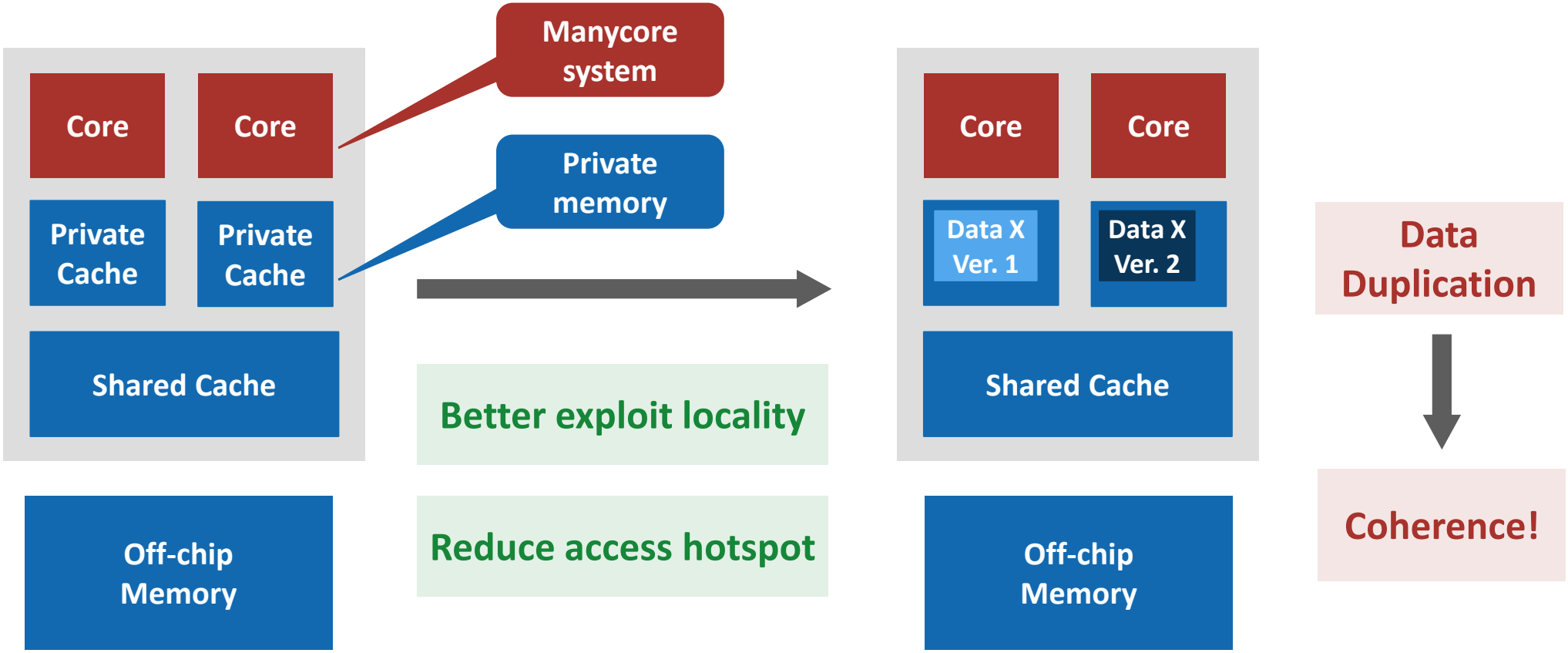
Unpredictability

Adapts to runtime-dependent behavior
Does not require knowledge on data location in advance

Cache-based Memory



Cache-based Memory



The Challenge of Cache

Coherence!

ACE (AXI Coherency Extensions)

AXI-channel based
Small-to-medium scale
Poor scalability

CHI (Coherent Hub Interface)

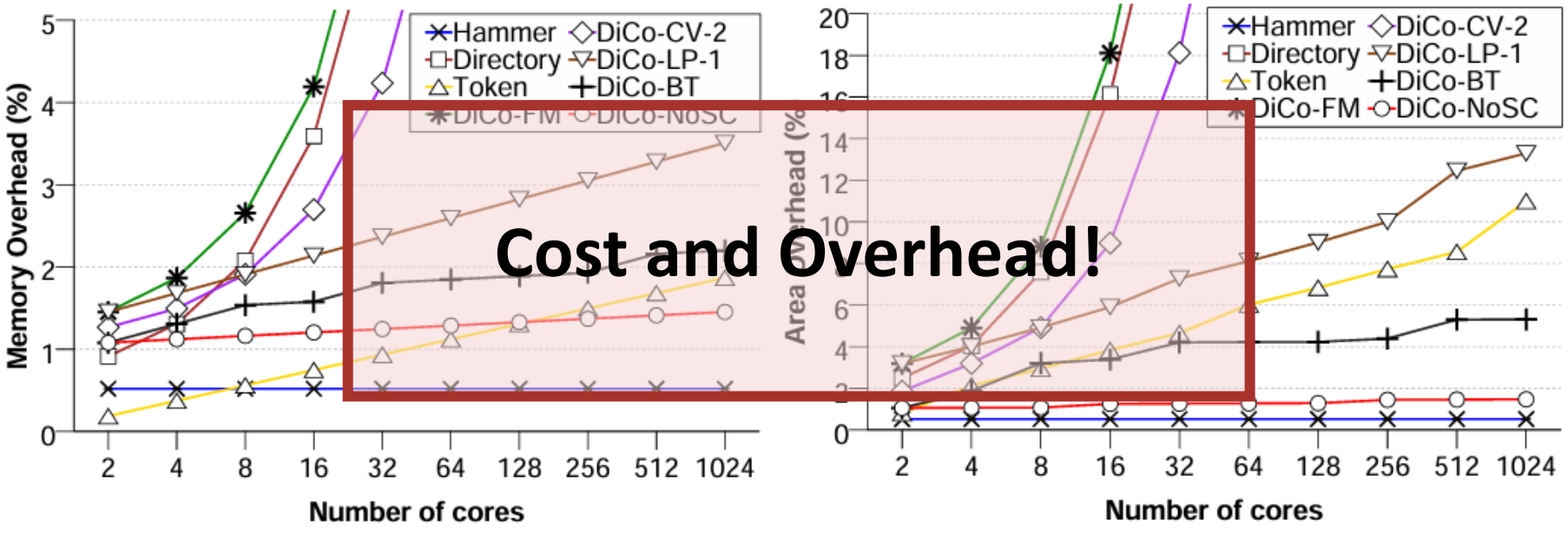
Highly scalable
Efficient bandwidth usage
Higher complexity
Larger area overhead

Complexity Overhead

Small-scale
Lightweight
Low-cost

The Challenge of Cache

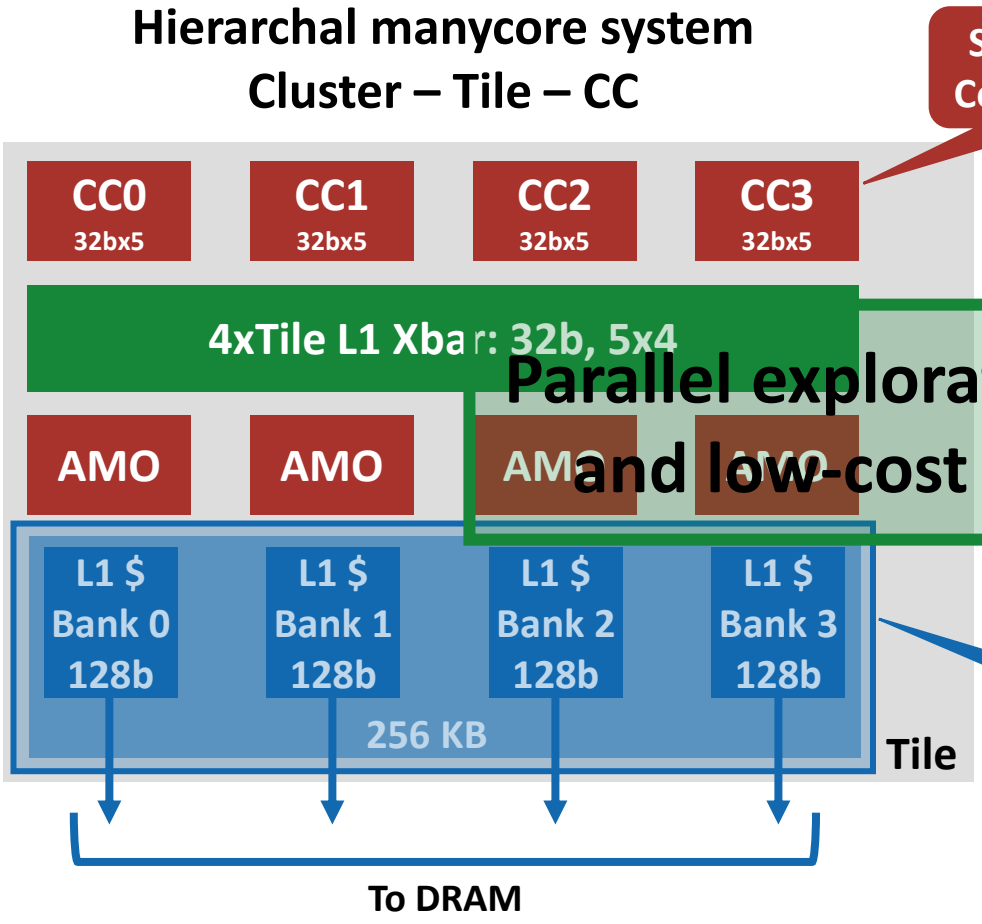
Overhead introduced by the cache coherence protocols.



(a) Overhead in terms of bits.

(b) Overhead in terms of area (mm^2).

CachePool



- **Shared Cache-Based Memory System**
  - **Fully shared, multi-banked data cache**
  - **Hardware-managed**
  - **Insitu-cache**
  - **Non-blocking cache that tolerates long-latency accesses DRAM**
  - **In-cacheline handling misses**

Parallel exploration of lightweight and low-cost private L1 cache

What about all those benefits?

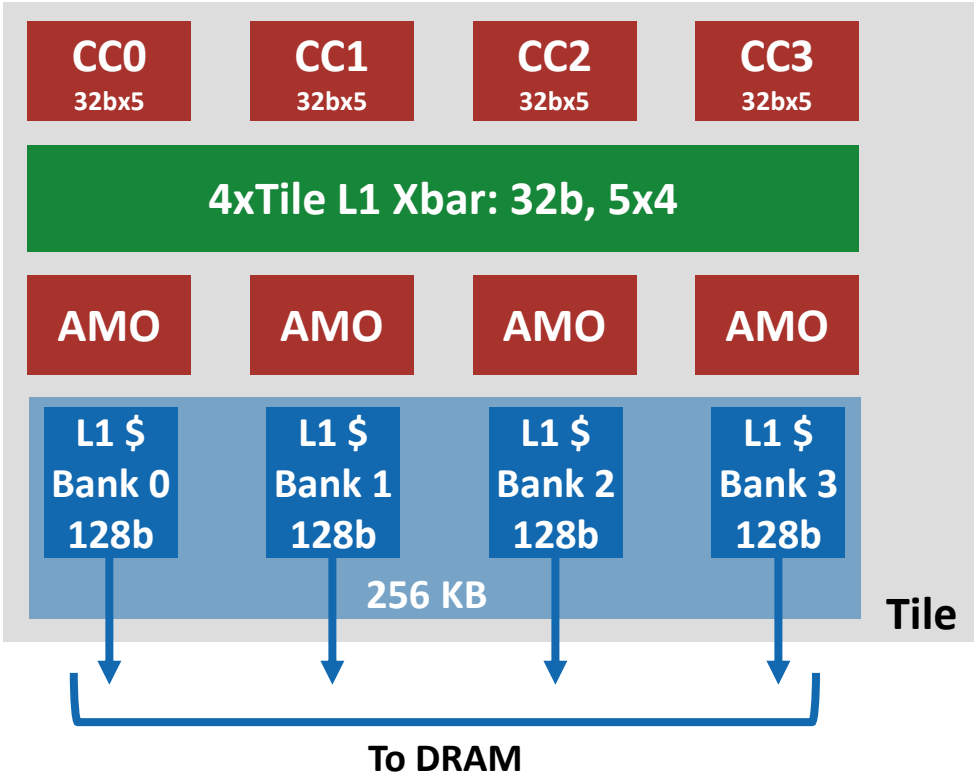
Contributions

- **Extend the cache hierarchy of CachePool**
  - Integrated write-through per-core private L1 data cache
- **Developed a light-weight coherence protocol**
  - L2 Tag-embedded coherence directory
  - Lightweight directory controller
  - Directory-based MESI protocol
- **Explored impact on performance and physical design**

Contributions

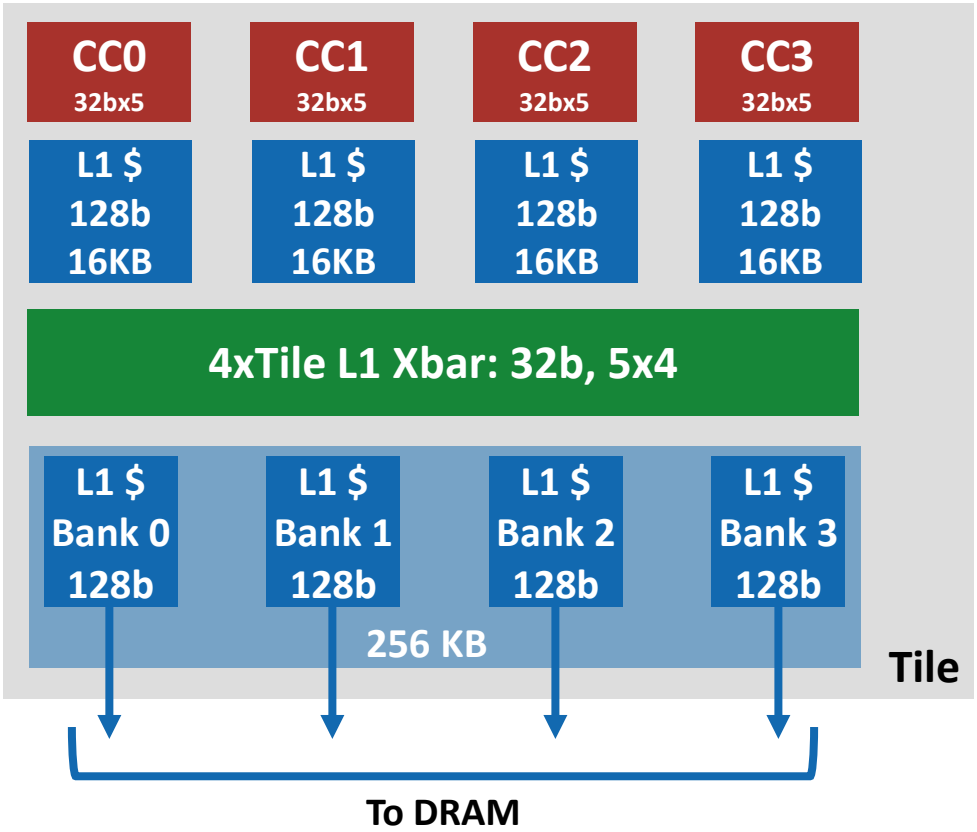
- **Extend the cache hierarchy of CachePool**
  - **Integrated write-through per-core private L1 data cache**
- **Developed a light-weight coherence protocol**
  - L2 Tag-embedded coherence directory
  - Lightweight directory controller
  - Directory-based MESI protocol
- **Explored impact on performance and physical design**

Private L1 Cache

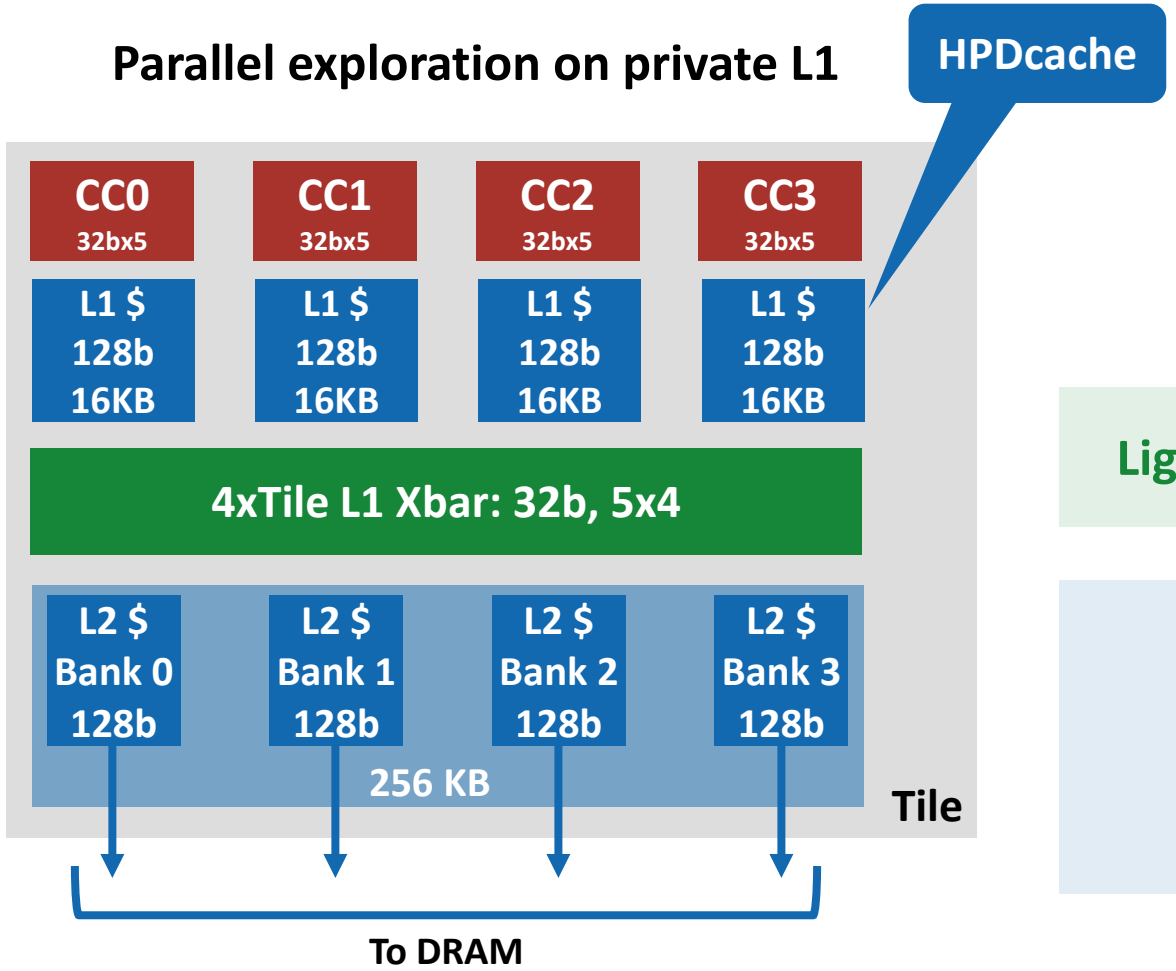


Private L1 Cache

Parallel exploration on private L1



Private L1 Cache



Non-blocking High-throughput
1-Cycle hit latency
Low-power design

Lightweight, Low-cost, Low-latency

Keep data closer to each CC

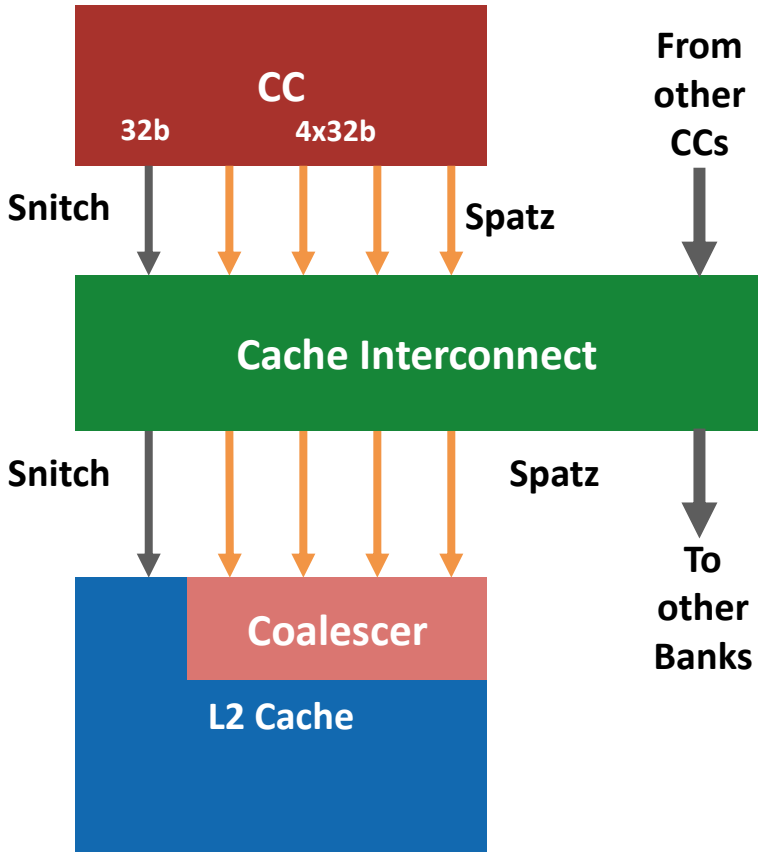
Write-through L1
Reduce coherence complexity

L1 Request Coalescing

Five 32-bit requests
1 Snitch + 4 Spatz

Spatz access continuous vector data

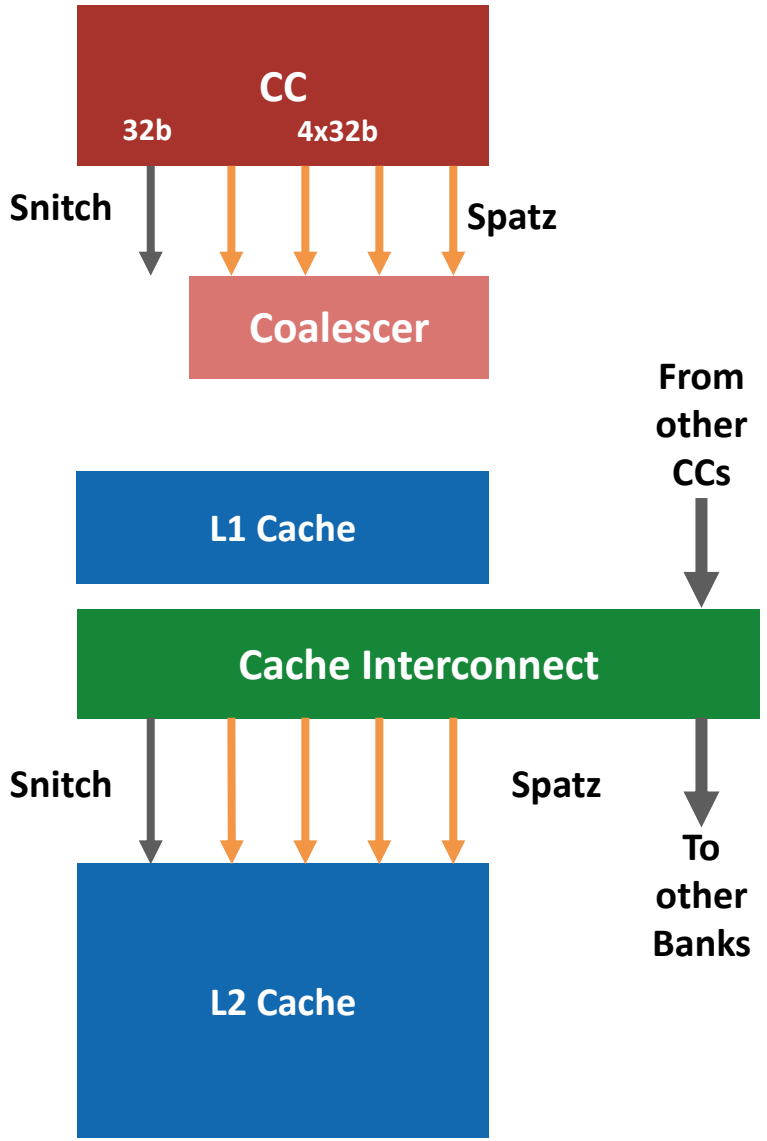
Coalesced by L2 controller



L1 Request Coalescing

Five 32-bit requests
1 Snitch + 4 Spatz

L1 cache introduced



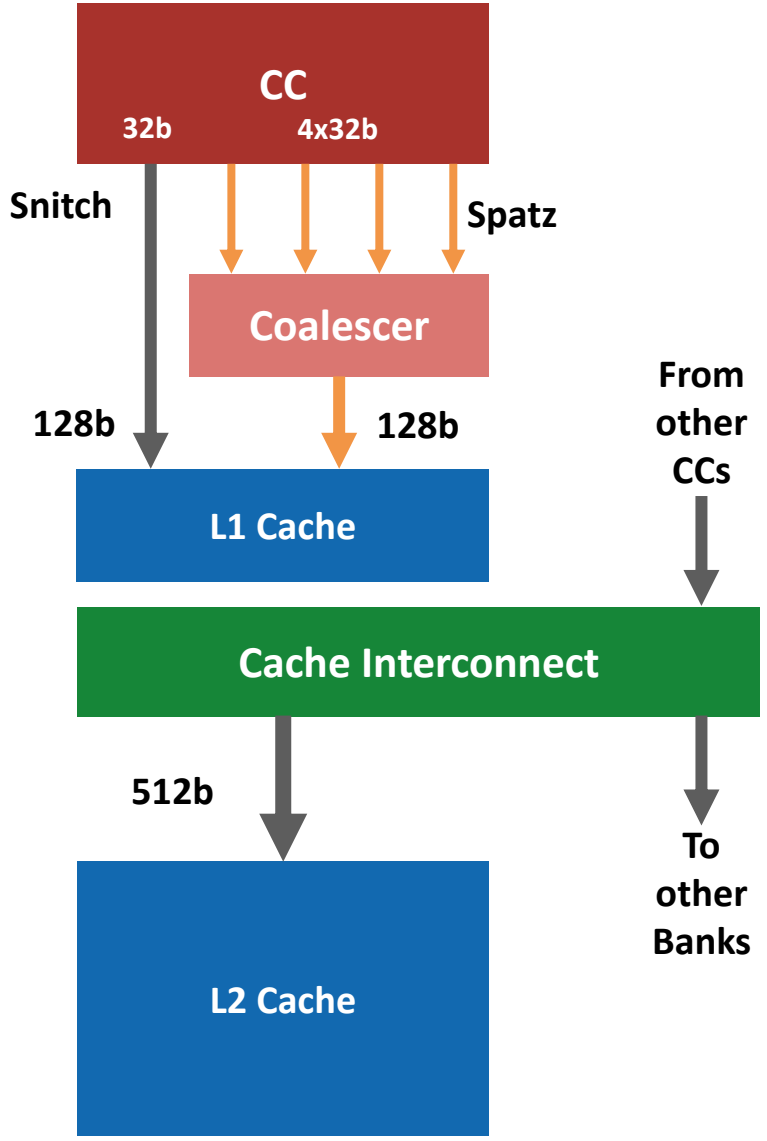
Coalescer moved forward

L1 Request Coalescing

Five 32-bit requests
1 Snitch + 4 Spatz

L1 cache introduced

Single request port
Cache line granularity



Coalescer moved forward

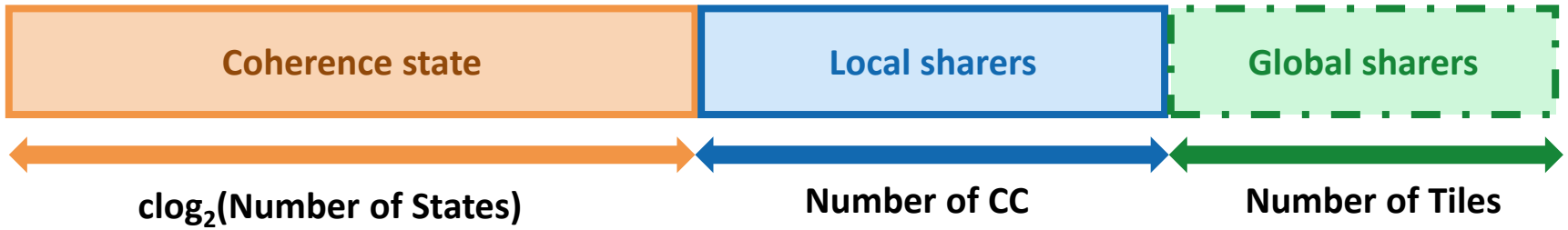
2-port L1
Snitch and Spatz

Contributions

- Extend the cache hierarchy of CachePool
  - Integrated write-through per-core private L1 data cache
- **Developed a light-weight coherence protocol**
  - **L2 Tag-embedded coherence directory**
  - Lightweight directory controller
  - Directory-based MESI protocol
- Explored impact on performance and physical design

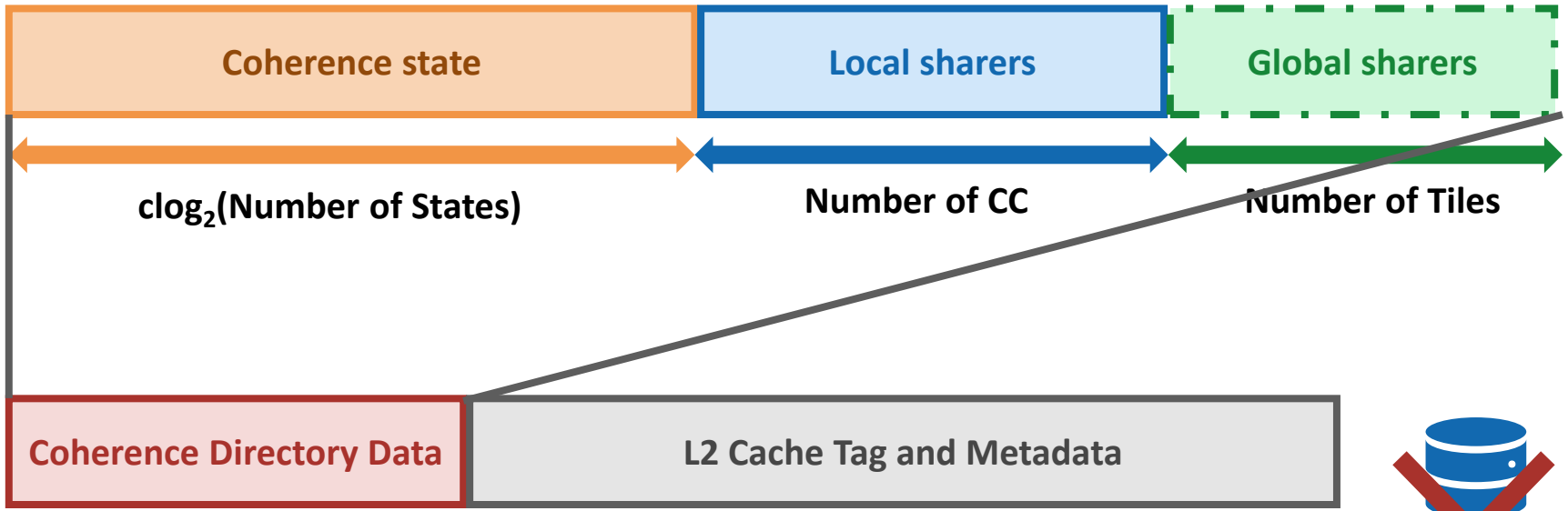
Coherence Directory Organization

Each L2 cache line must track



Coherence Directory Organization

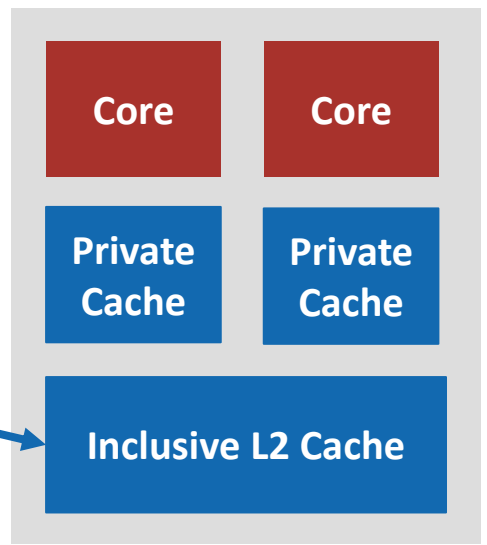
Each L2 cache line must track



Inclusive directory embedded in the inclusive L2

L2 as natural coordination point for coherence

Separate directory

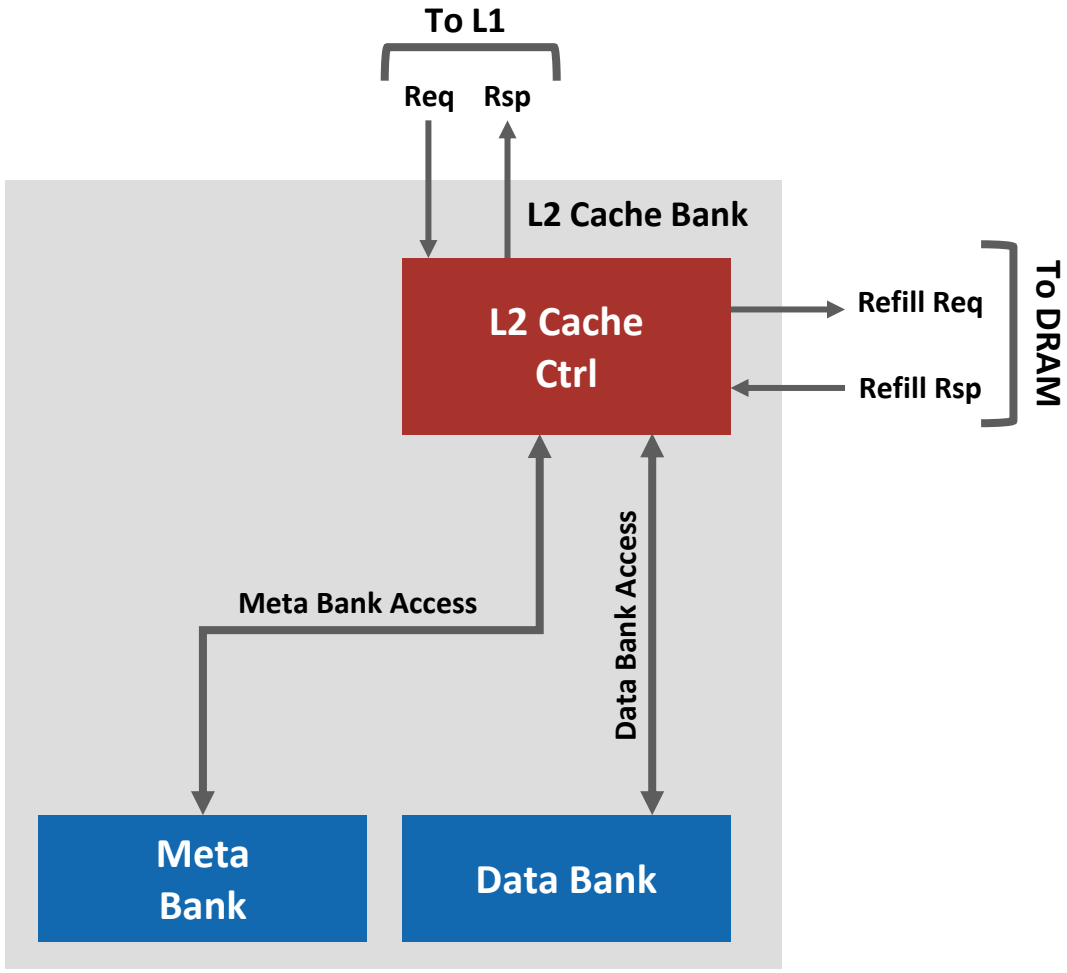


Lightweight, Low-cost

Contributions

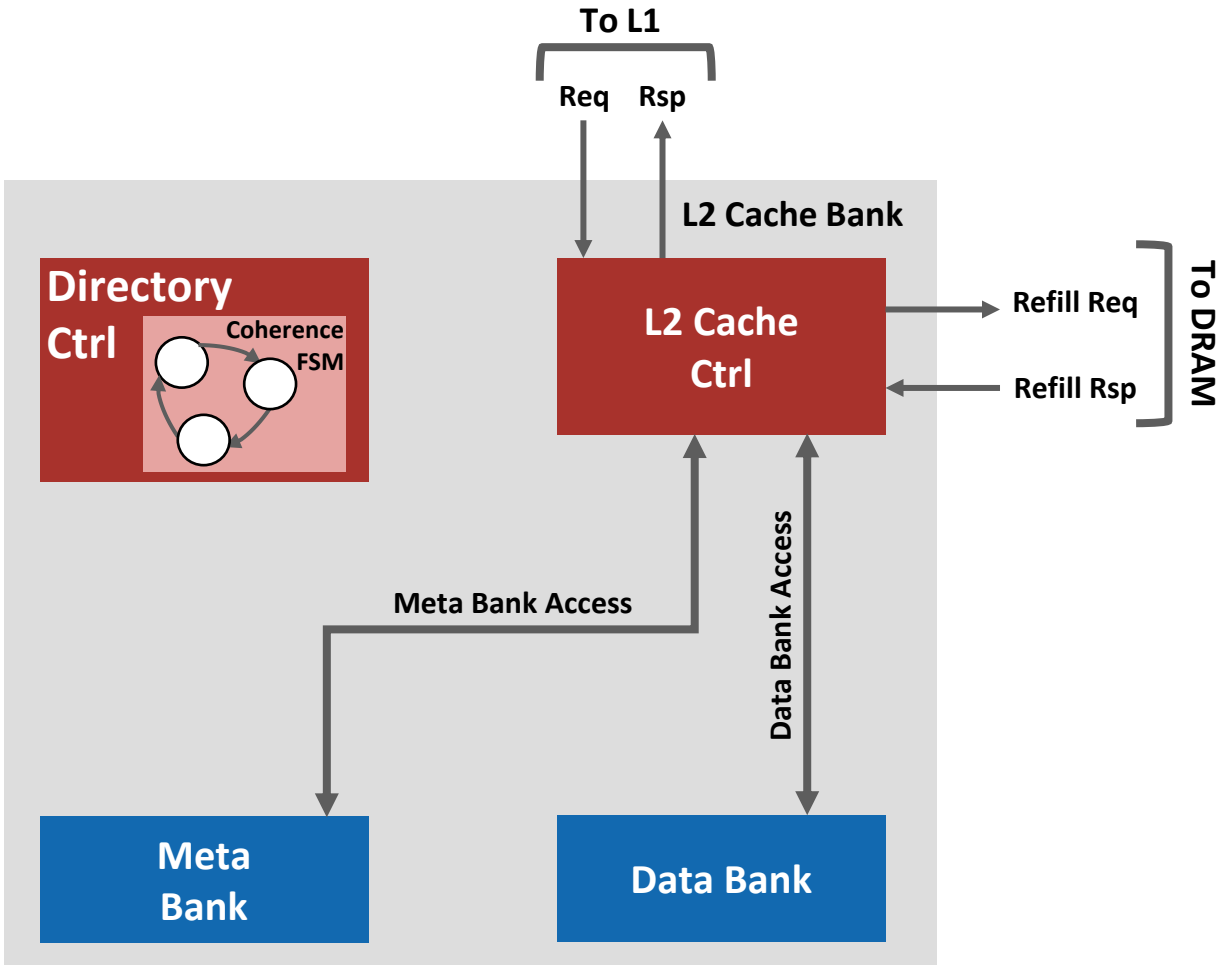
- Extend the cache hierarchy of CachePool
  - Integrated write-through per-core private L1 data cache
- **Developed a light-weight coherence protocol**
  - L2 Tag-embedded coherence directory
  - **Lightweight directory controller**
  - Directory-based MESI protocol
- Explored impact on performance and physical design

Directory Controller



Directory Controller

Each L2 bank has its directory controller



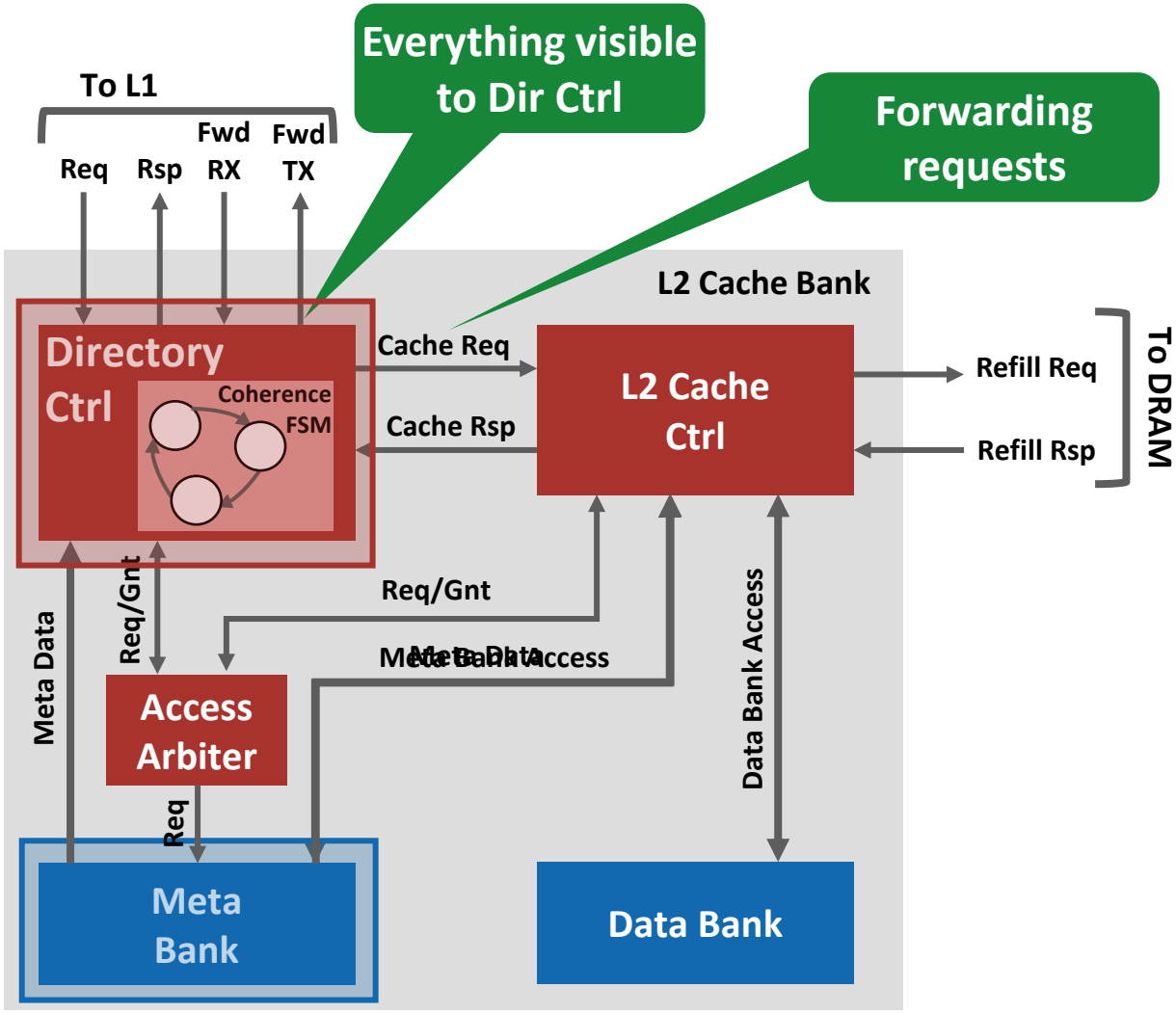
Directory Controller

Cache and coherence traffics

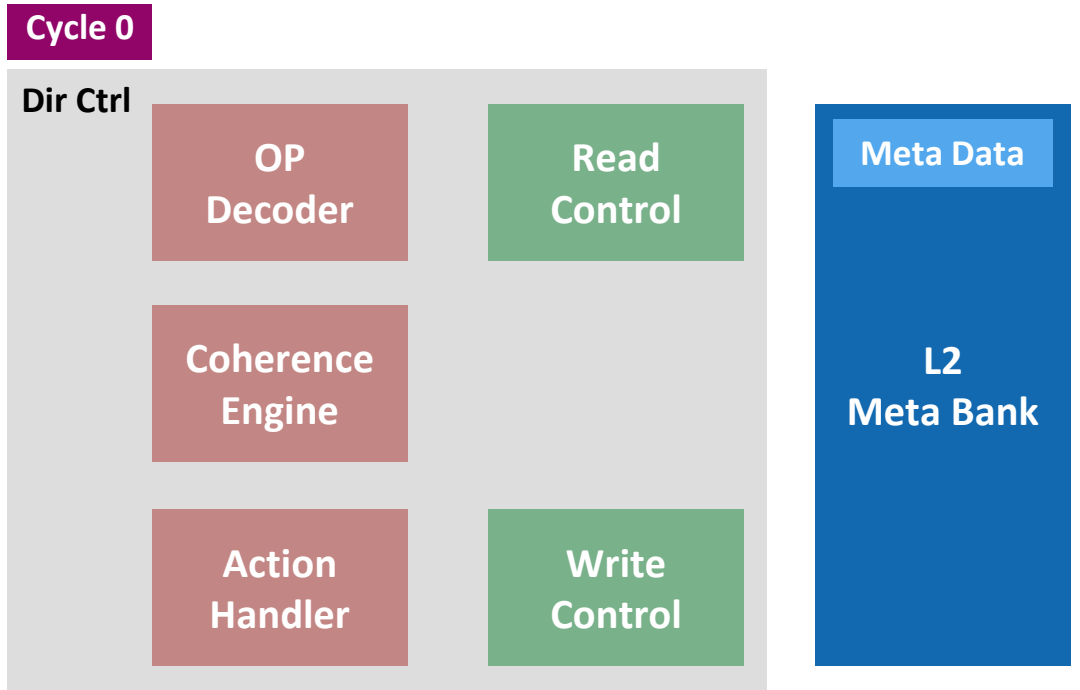
Each L2 bank has its directory controller

Request buffered and serialized

Meta bank managed together

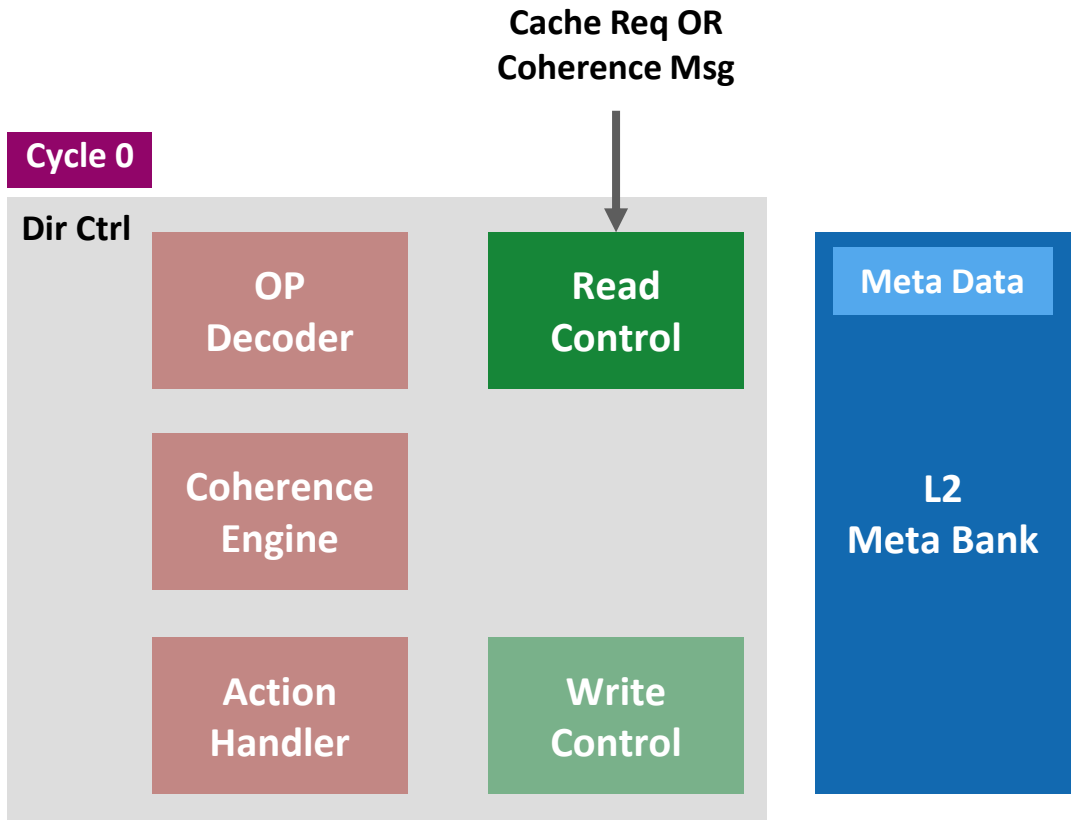


Directory Controller



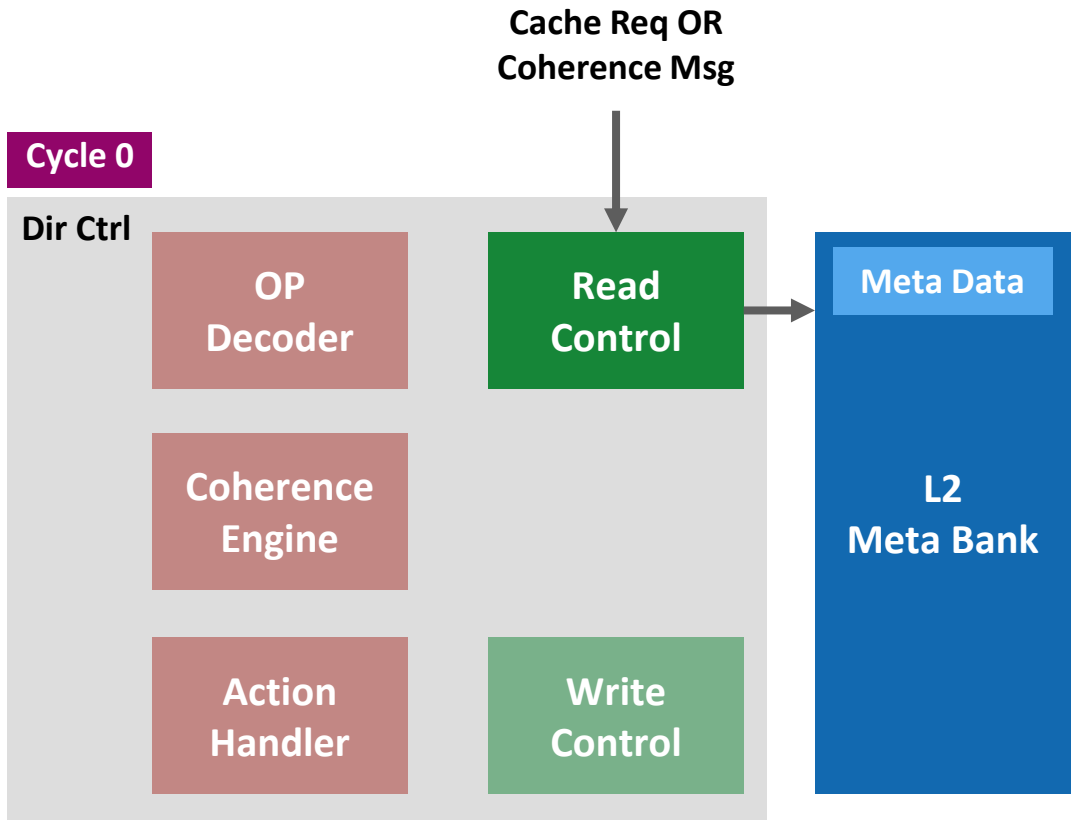
Directory Controller

1. **Incoming cache requests and coherence messages**



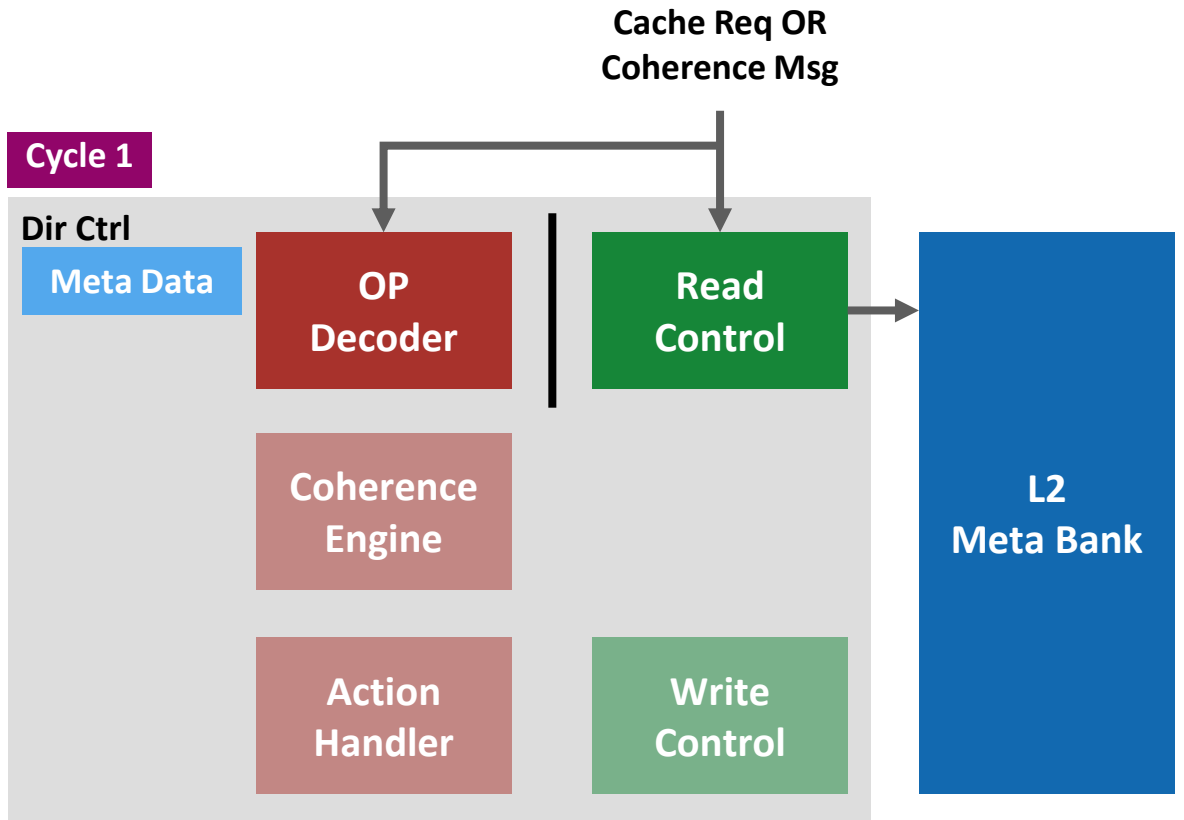
Directory Controller

1. Incoming cache requests and coherence messages
2. **Read from meta bank**



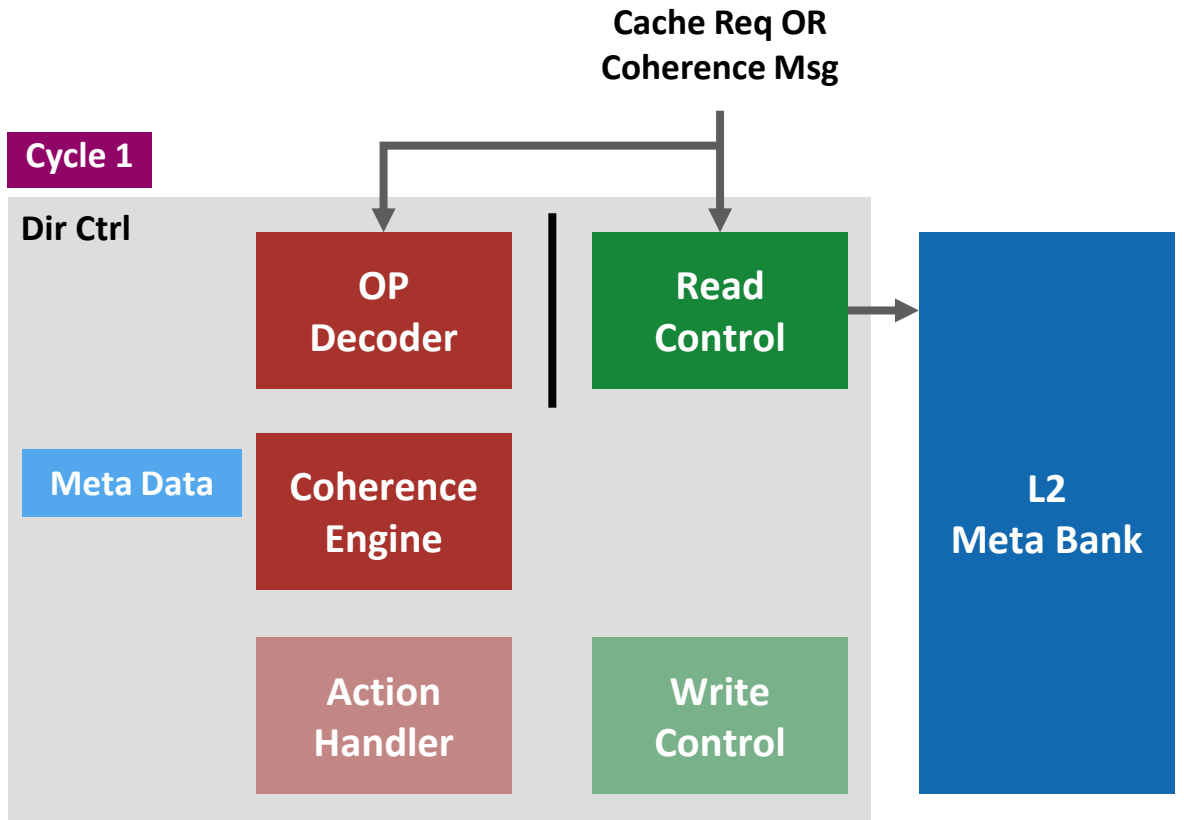
Directory Controller

1. Incoming cache requests and coherence messages
2. Read from meta bank
3. **Coherence OP decode**



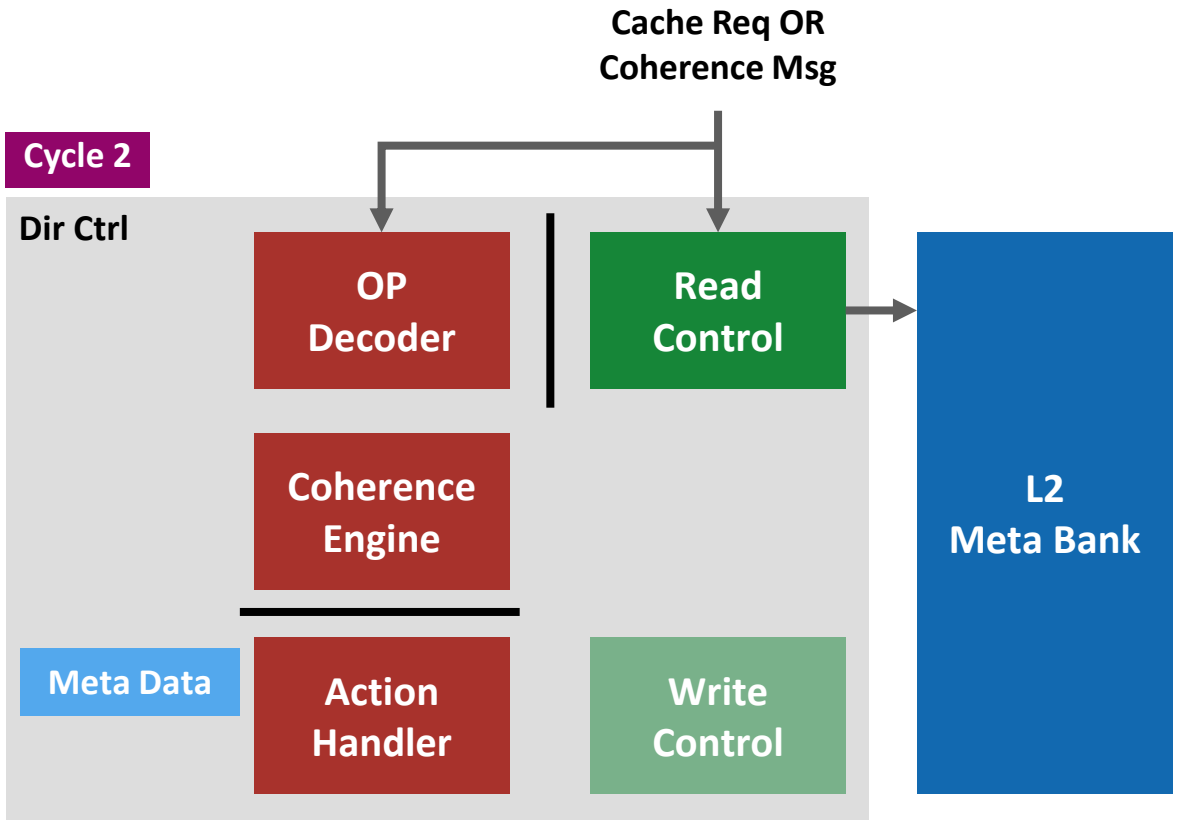
Directory Controller

1. Incoming cache requests and coherence messages
2. Read from meta bank
3. Coherence OP decode
4. **Coherence Engine**
   a) **New coherence state**
   b) **New sharer list**
   c) **Actions**



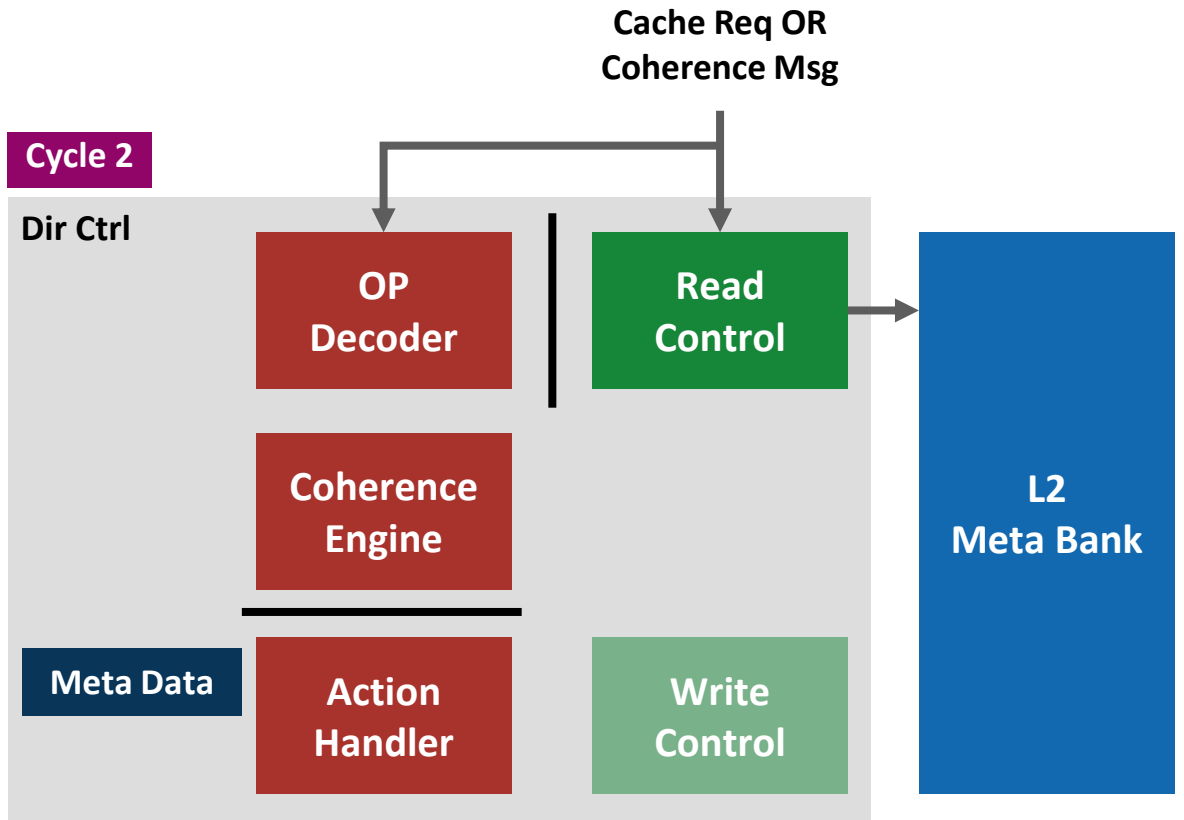
Directory Controller

1. Incoming cache requests and coherence messages
2. Read from meta bank
3. Coherence OP decode
4. Coherence Engine
   a) New coherence state
   b) New sharer list
   c) Actions
5. **Action Handling**



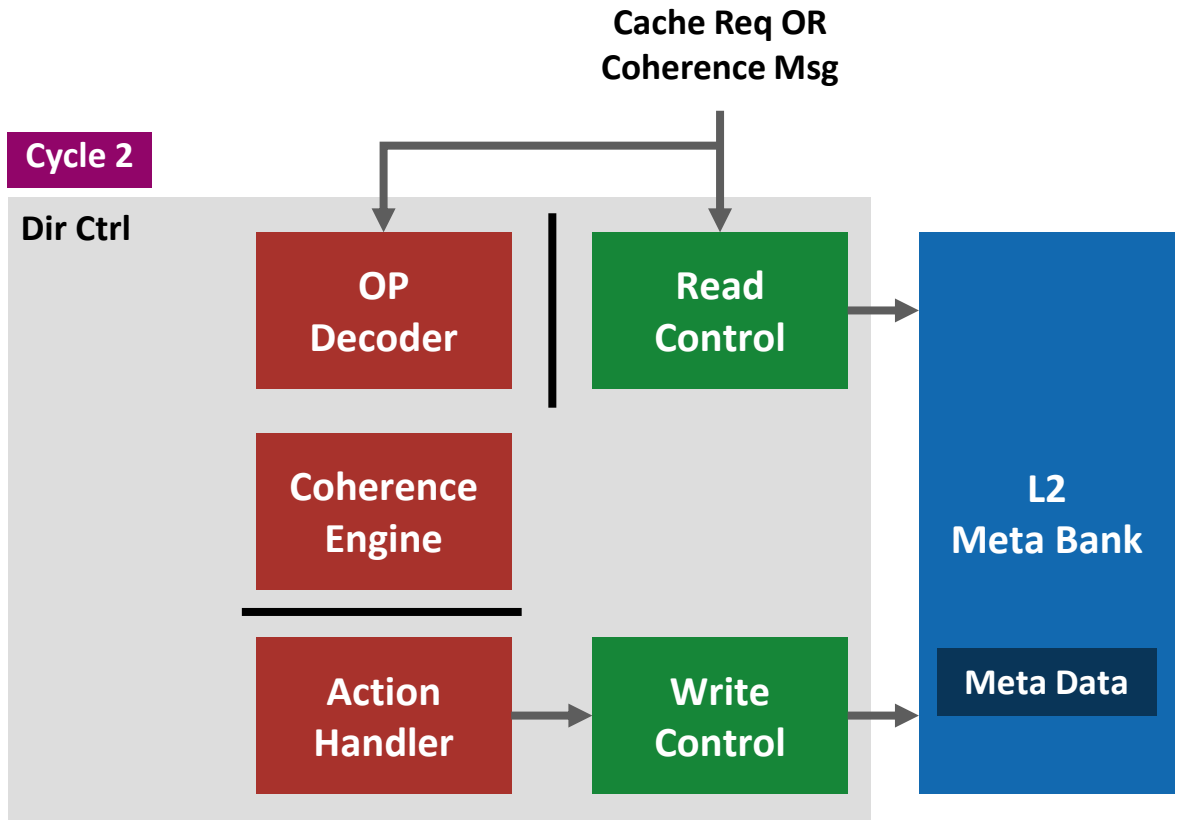
Directory Controller

1. Incoming cache requests and coherence messages
2. Read from meta bank
3. Coherence OP decode
4. Coherence Engine
   a) New coherence state
   b) New sharer list
   c) Actions
5. **Action Handling**
   a) **Update coherence meta data**



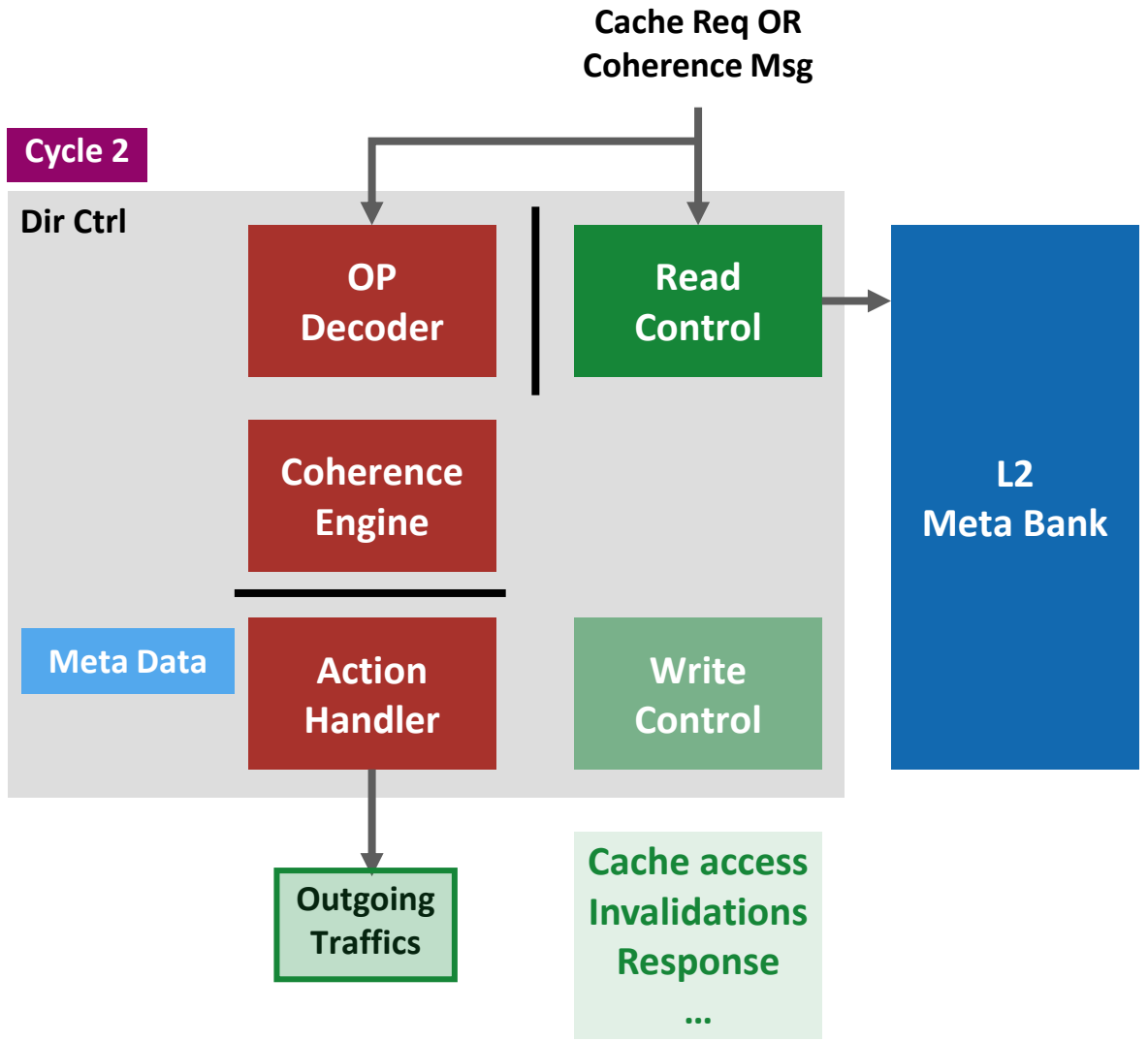
Directory Controller

1. Incoming cache requests and coherence messages
2. Read from meta bank
3. Coherence OP decode
4. Coherence Engine
   a) New coherence state
   b) New sharer list
   c) Actions
5. **Action Handling**
   a) **Update coherence meta data**



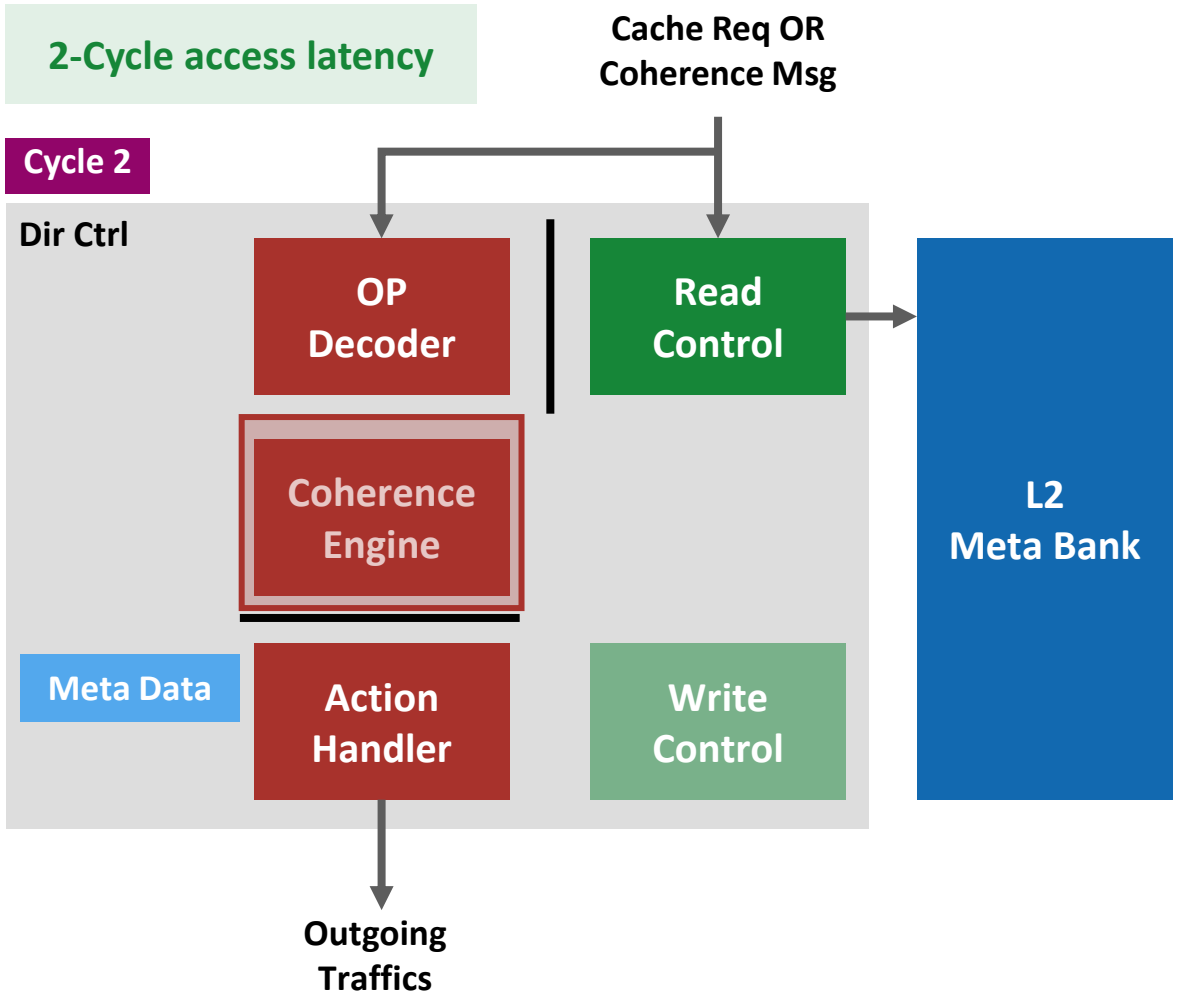
Directory Controller

1. Incoming cache requests and coherence messages
2. Read from meta bank
3. Coherence OP decode
4. Coherence Engine
   a) New coherence state
   b) New sharer list
   c) Actions
5. **Action Handling**
   a) Update coherence meta data
   b) **Outbound traffics**



Directory Controller

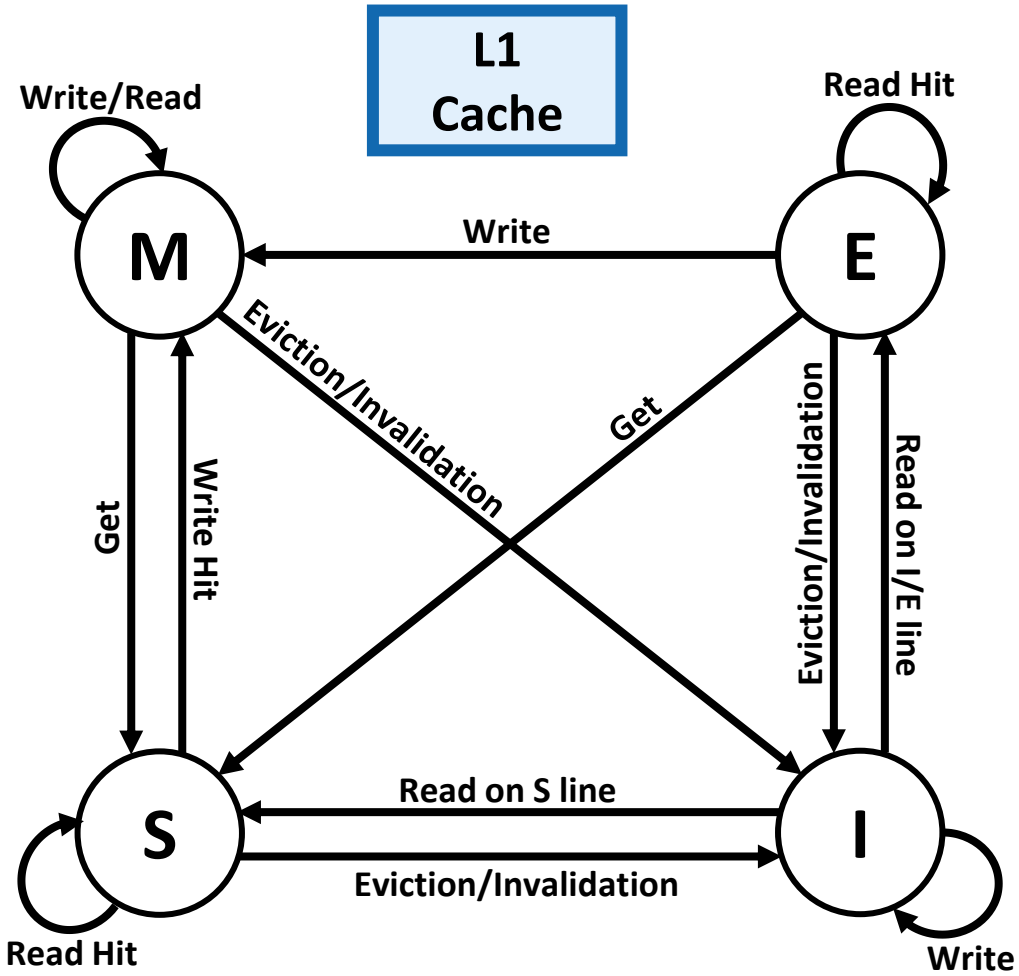
1. Incoming cache requests and coherence messages
2. Read from meta bank
3. Coherence OP decode
4. Coherence Engine
   a) New coherence state
   b) New sharer list
   c) Actions
5. Action Handling
   a) Update coherence meta data
   b) Outbound traffics

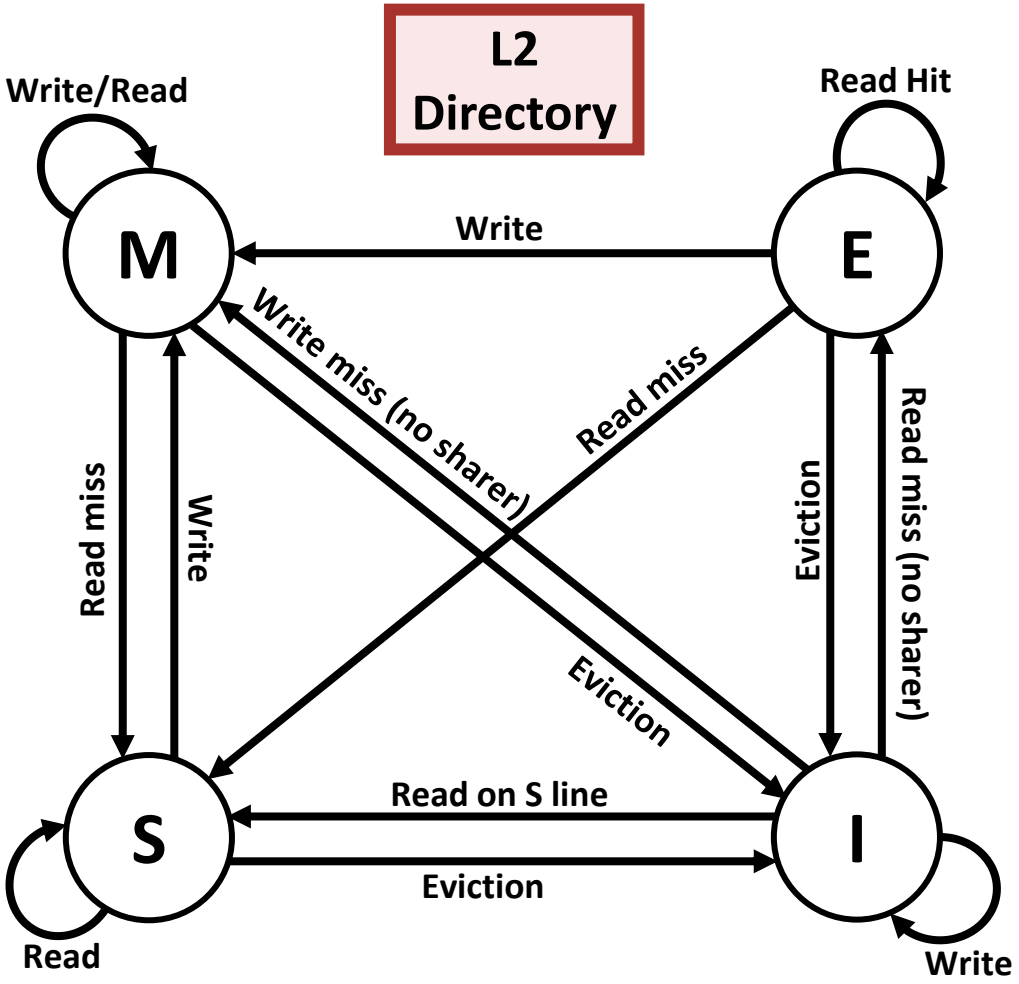


Contributions

- Extend the cache hierarchy of CachePool
  - Integrated write-through per-core private L1 data cache
- **Developed a light-weight coherence protocol**
  - L2 Tag-embedded coherence directory
  - Lightweight directory controller
  - **Directory-based MESI protocol**
- Explored impact on performance and physical design

Coherence Protocol



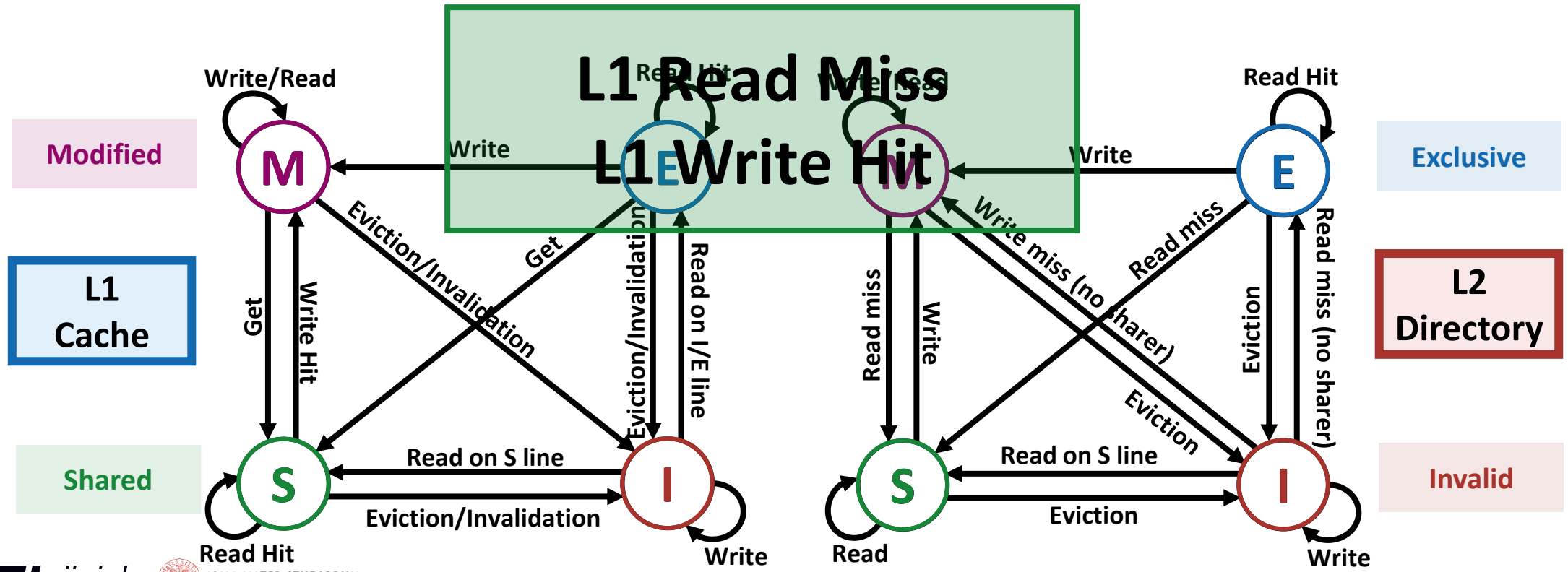


Coherence Protocol

Directory-based, lightweight MESI protocol

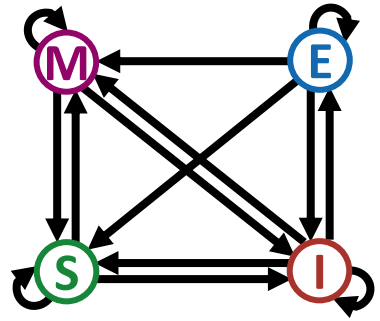
E-State to reduce unnecessary coherence operation

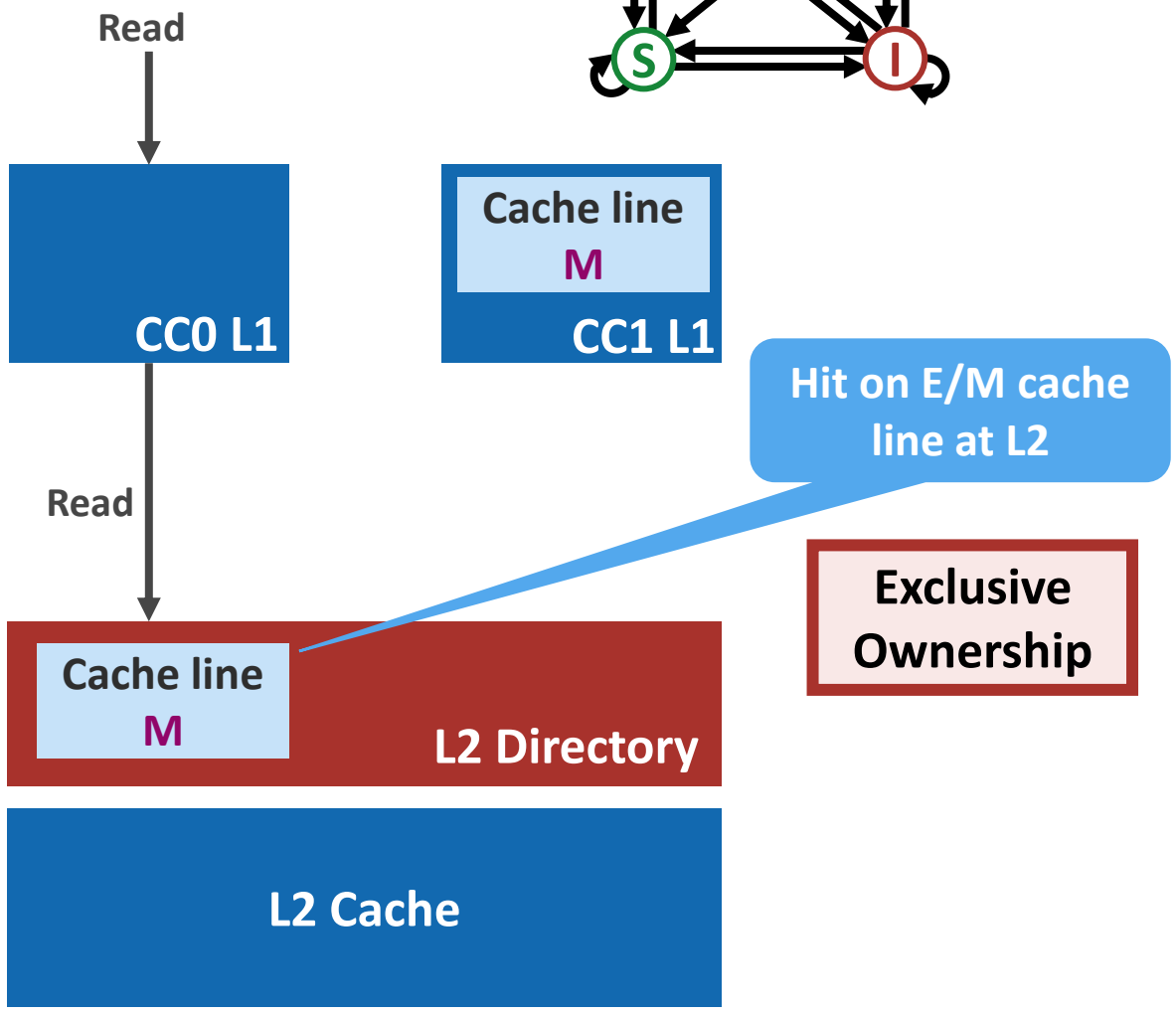
M-State meaningful for our cache with write buffer



L1 Read Miss

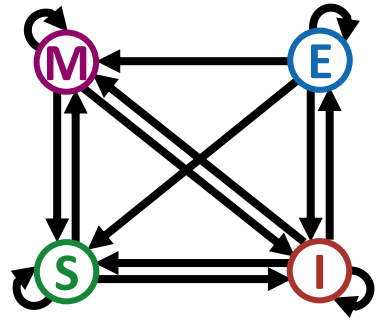
1. **Read miss at L1 level**

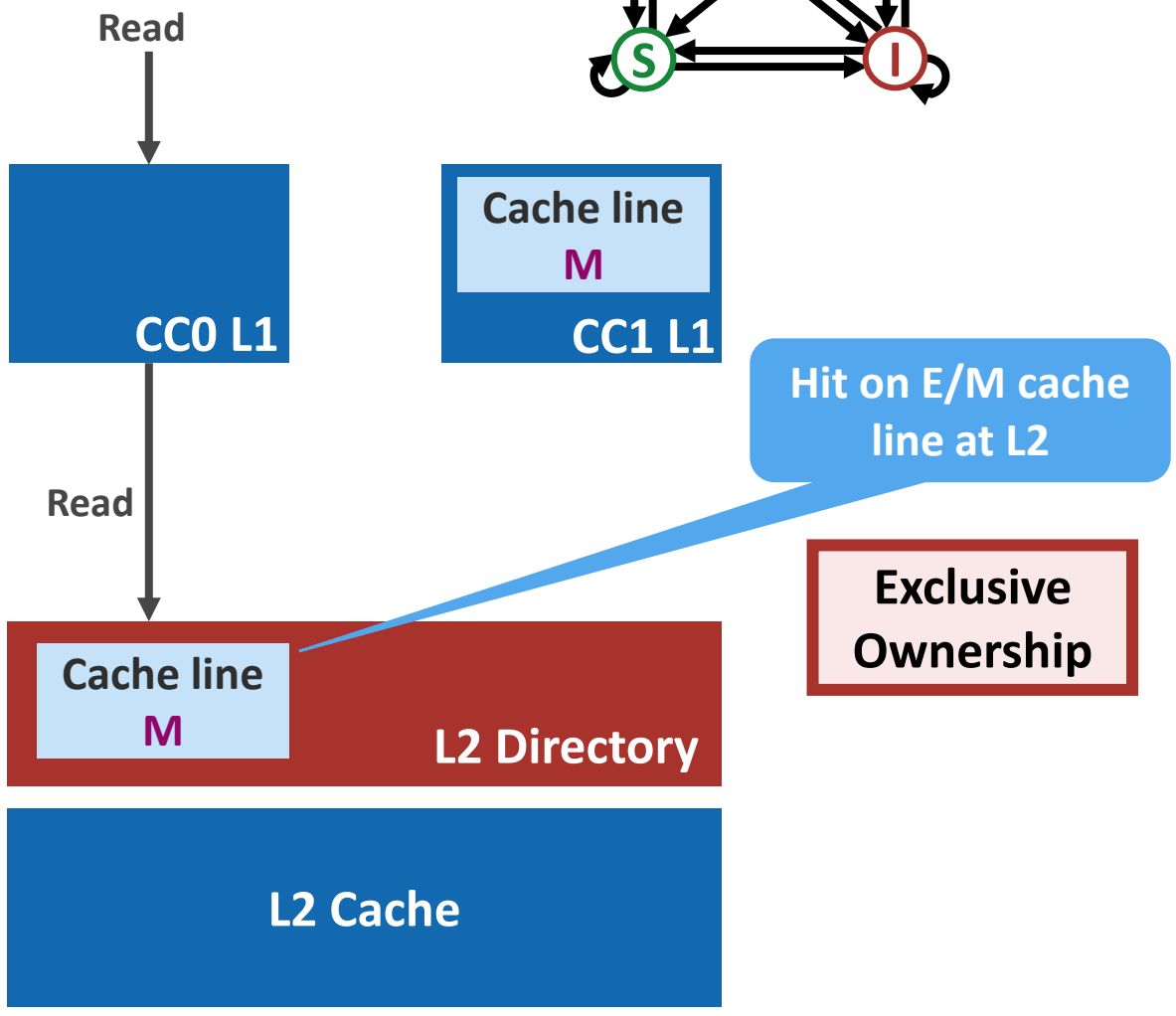




L1 Read Miss

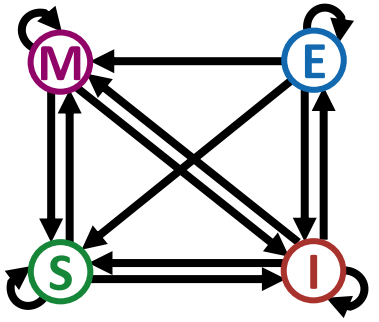
1. Read miss at L1 level
2. Questions
   - What is the latest state of the cache line?
   - Is the data up-to-date?

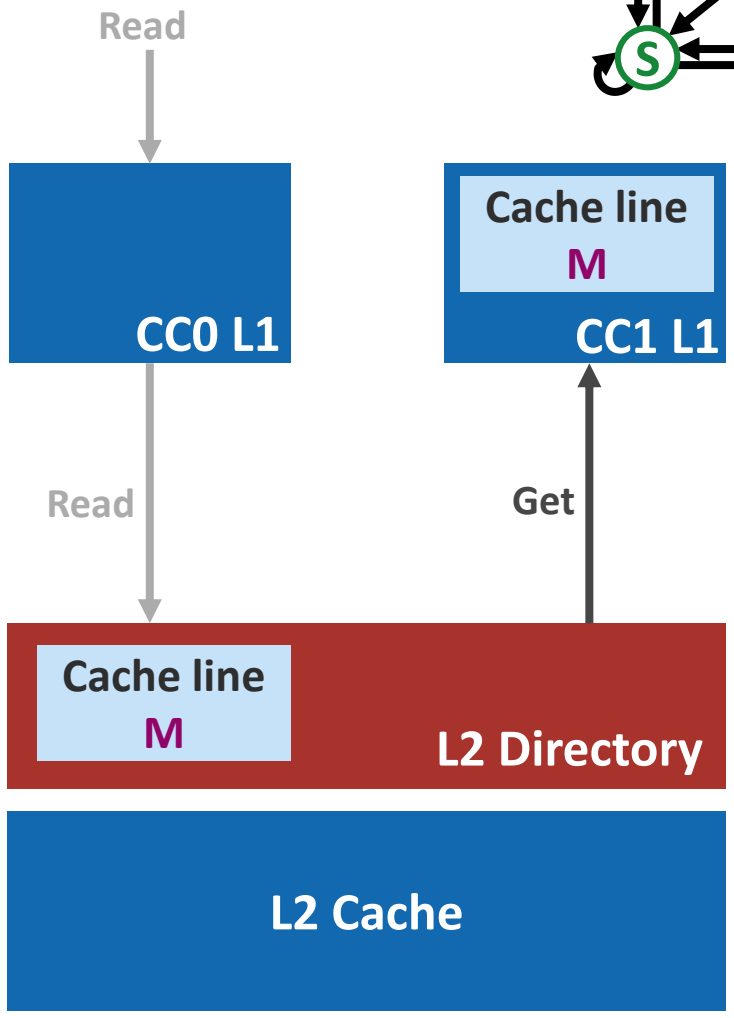




L1 Read Miss

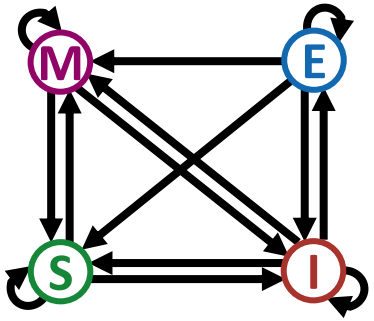
1. Read miss at L1 level
2. Questions
   - What is the latest state of the cache line?
   - Is the data up-to-date?
3. **Send Get to the probe the owner**

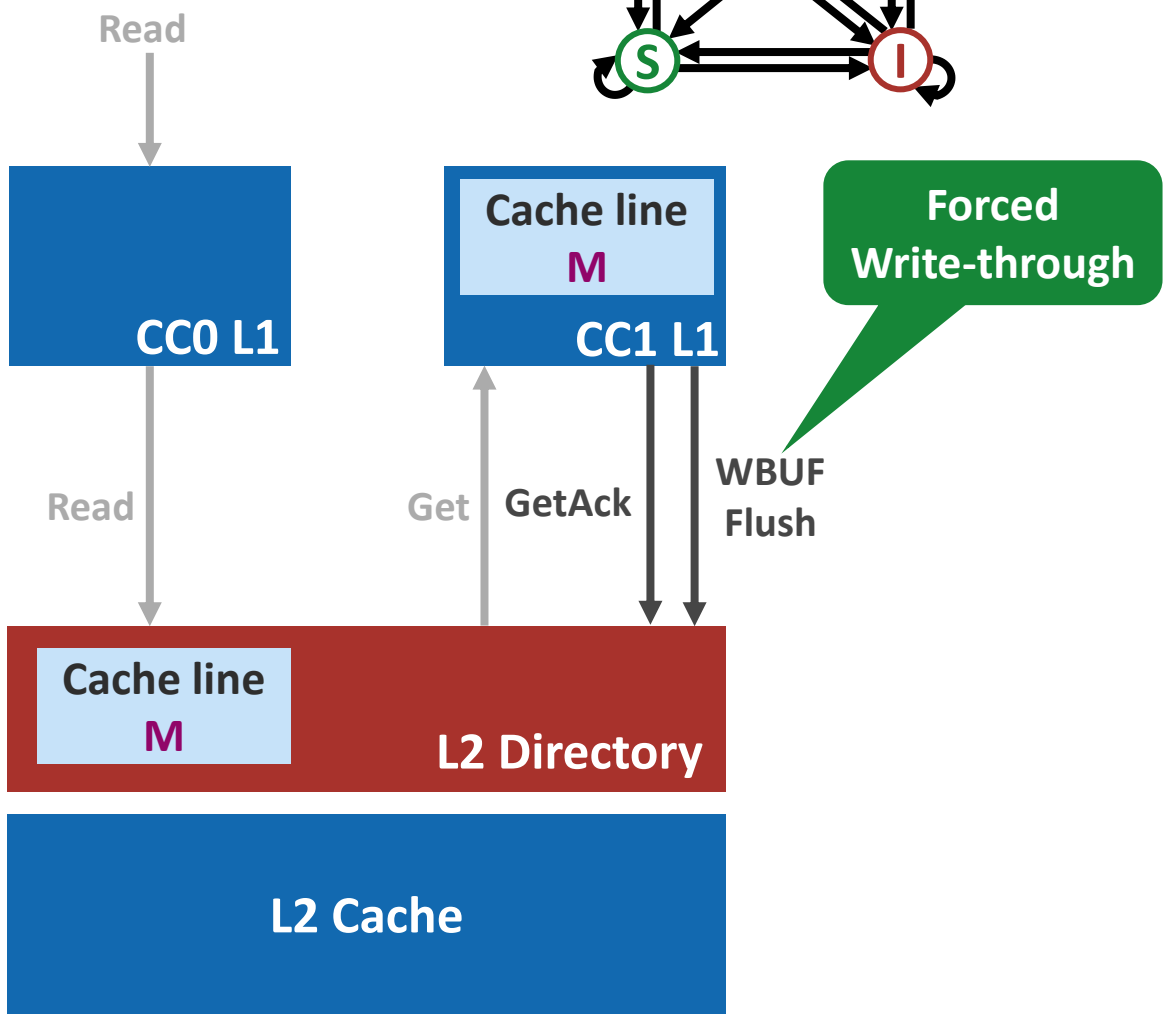




L1 Read Miss

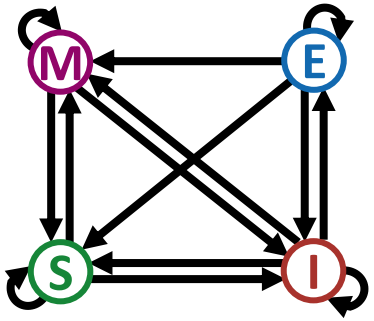
1. Read miss at L1 level
2. Questions
   - What is the latest state of the cache line?
   - Is the data up-to-date?
3. Send Get to the probe the owner
4. **Owner will**
   1. **Respond with GetAck**
   2. **Flush write buffer**

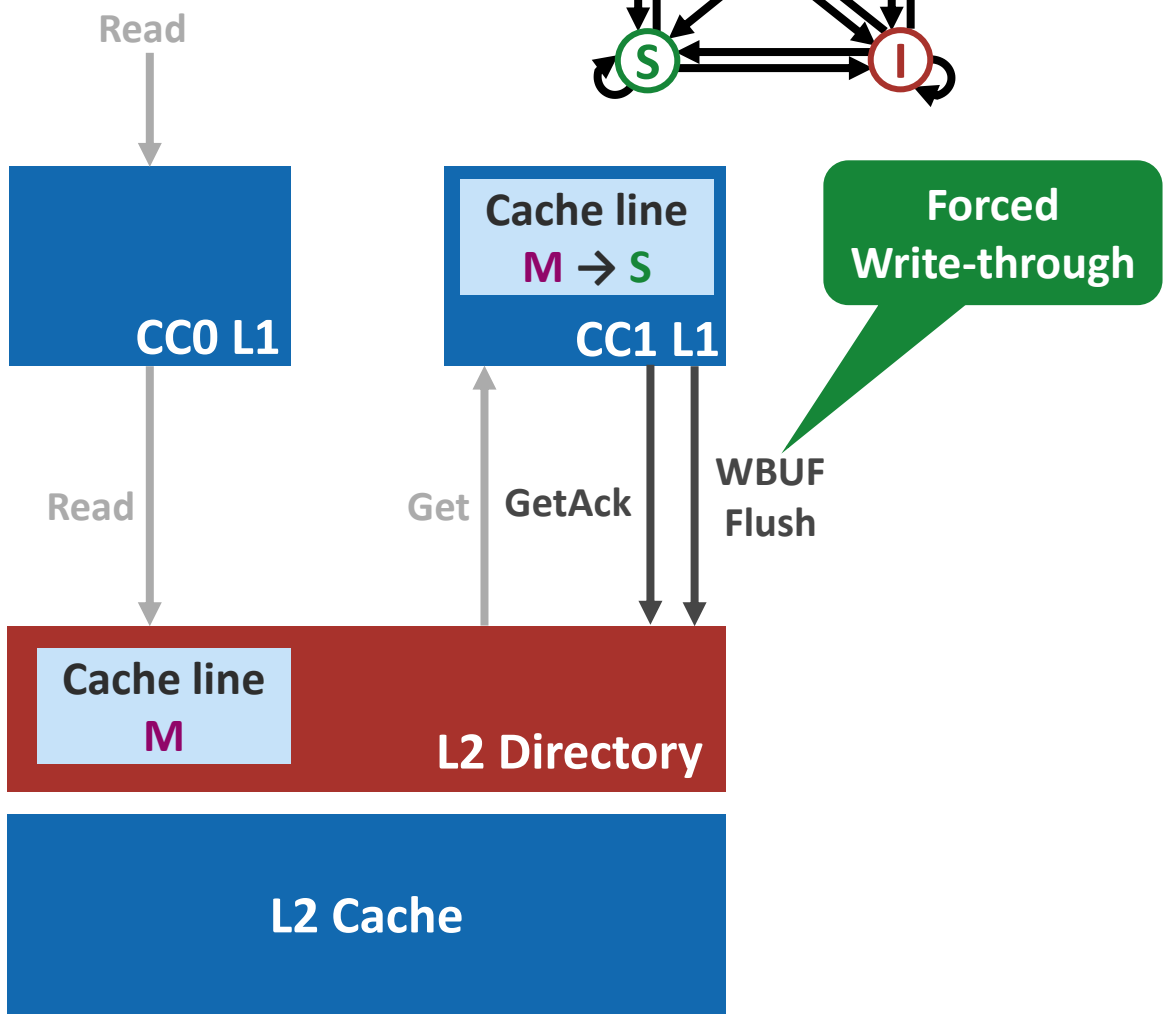




L1 Read Miss

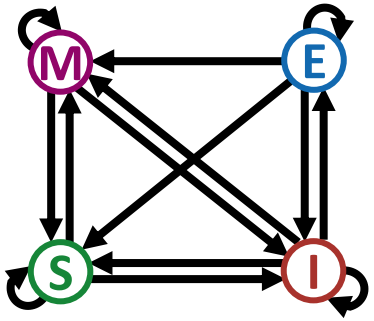
1. Read miss at L1 level
2. Questions
   - What is the latest state of the cache line?
   - Is the data up-to-date?
3. Send Get to the probe the owner
4. **Owner will**
   1. **Respond with GetAck**
   2. **Flush write buffer**
   3. **Update state**

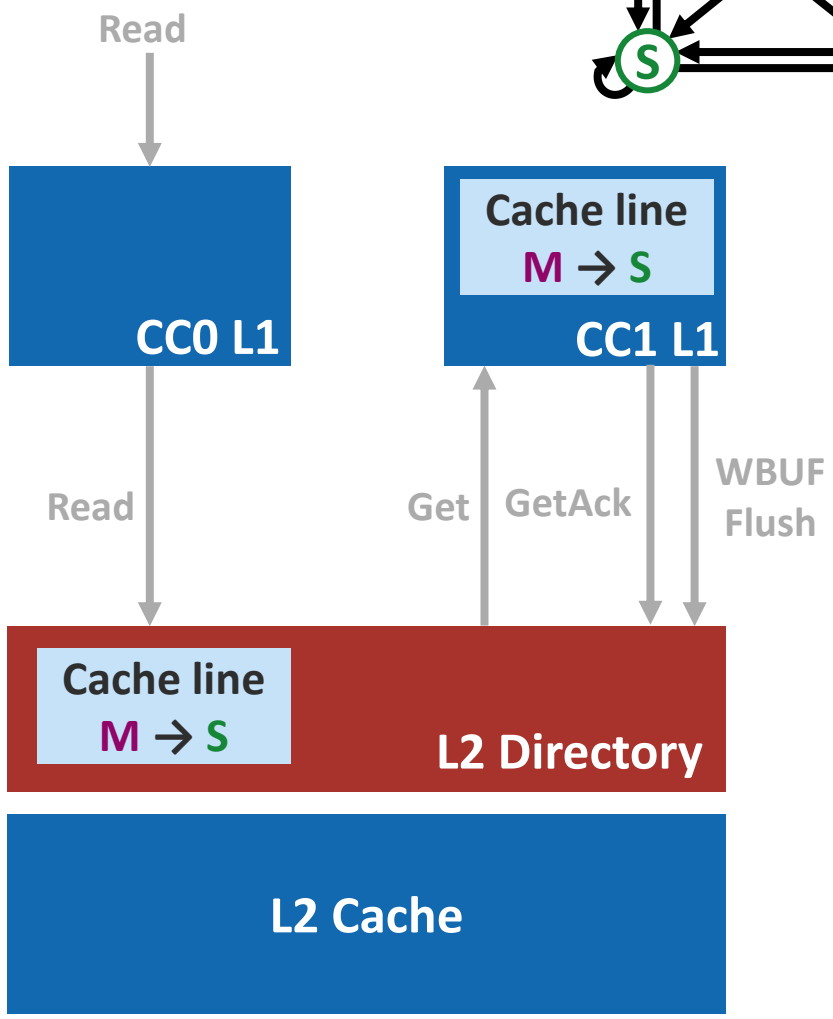




L1 Read Miss

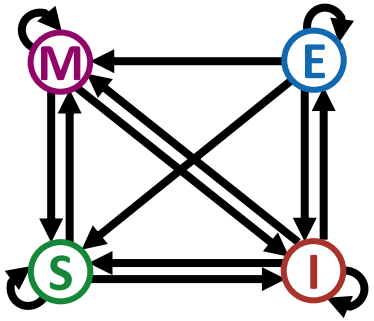
1. Read miss at L1 level
2. Questions
   - What is the latest state of the cache line?
   - Is the data up-to-date?
3. Send Get to the probe the owner
4. Owner will
   1. Respond with GetAck
   2. Flush write buffer
   3. Update state
5. **Directory then**
   1. **Update state**

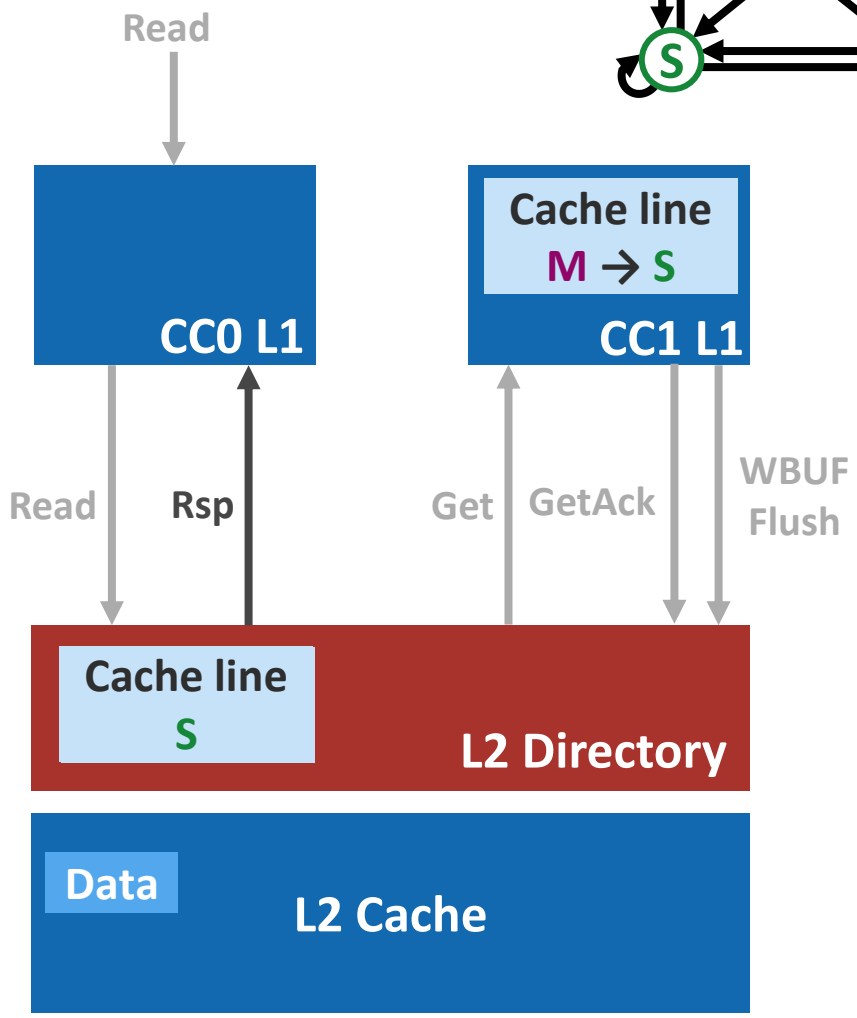




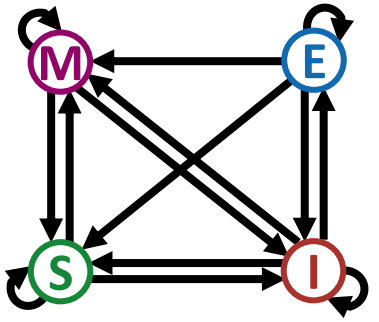
L1 Read Miss

1. Read miss at L1 level
2. Questions
   - What is the latest state of the cache line?
   - Is the data up-to-date?
3. Send Get to the probe the owner
4. Owner will
   1. Respond with GetAck
   2. Flush write buffer
   3. Update state
5. **Directory then**
   1. **Update state**
   2. **Respond to the read request**



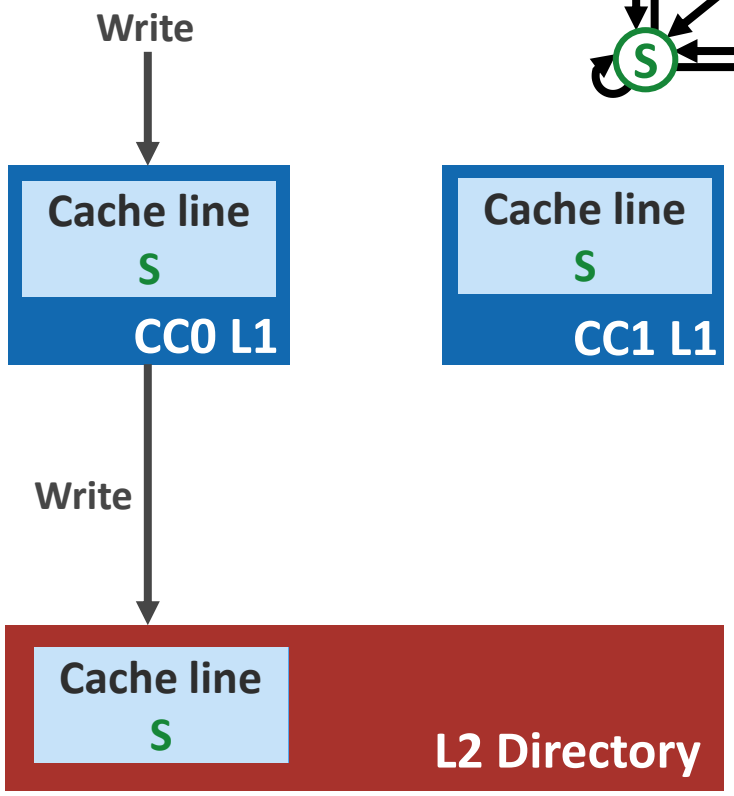


L1 Write Hit on Shared Line

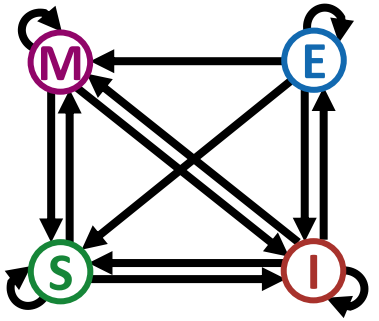


Shared! L1 cannot commit immediately!

1. **Directory gets informed**

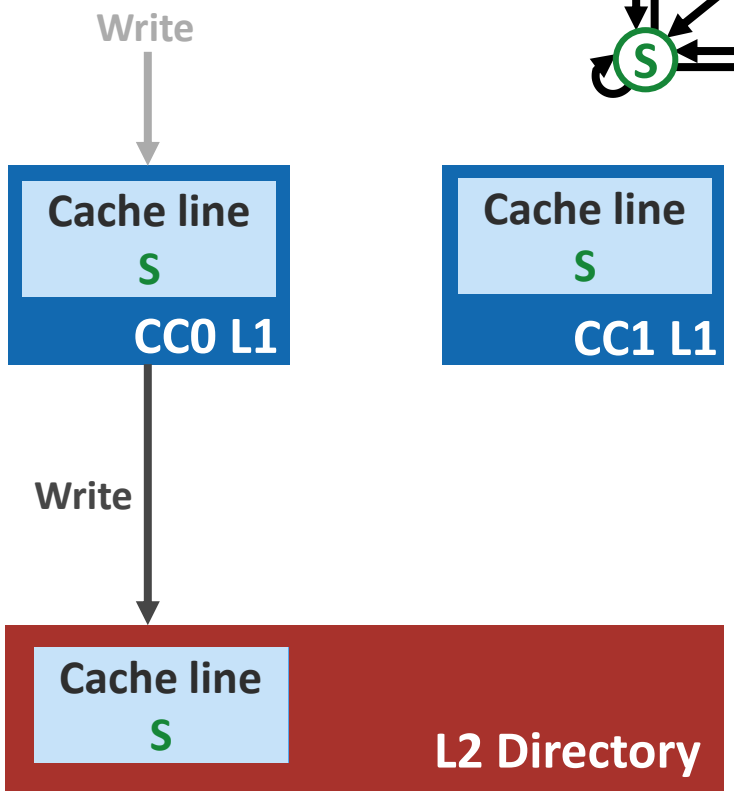


L1 Write Hit on Shared Line

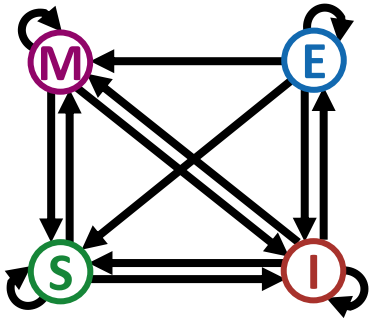


Shared! L1 cannot commit immediately!

1. Directory gets informed
2. **Directory**
   1. **Update coherence state and sharer list**

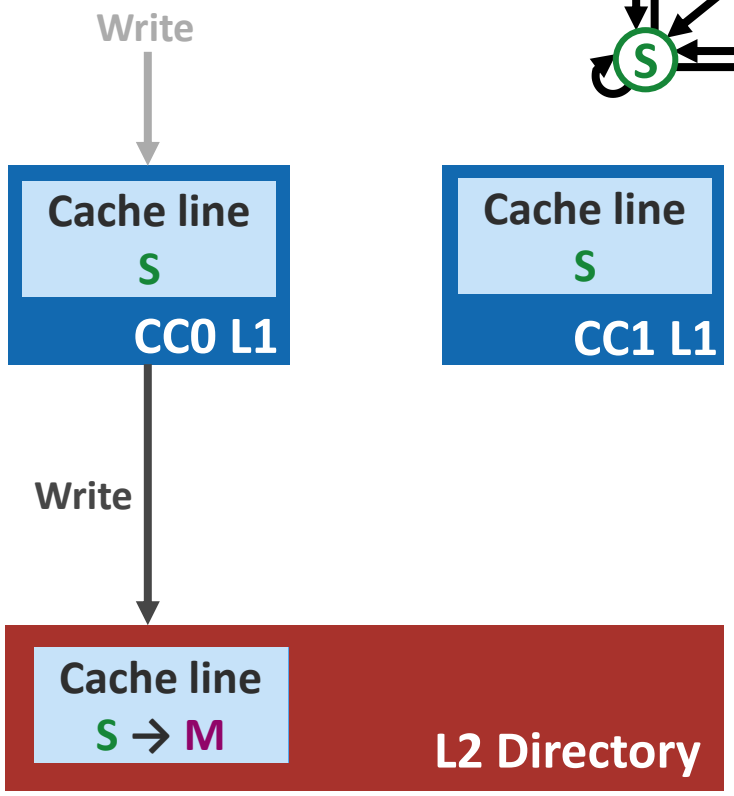


L1 Write Hit on Shared Line

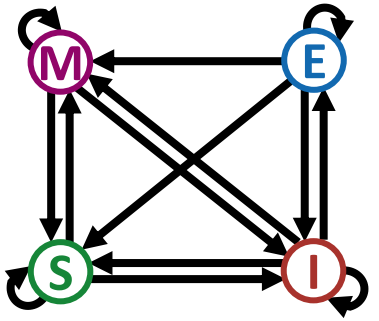


Shared! L1 cannot commit immediately!

1. Directory gets informed
2. **Directory**
   1. **Update coherence state and sharer list**

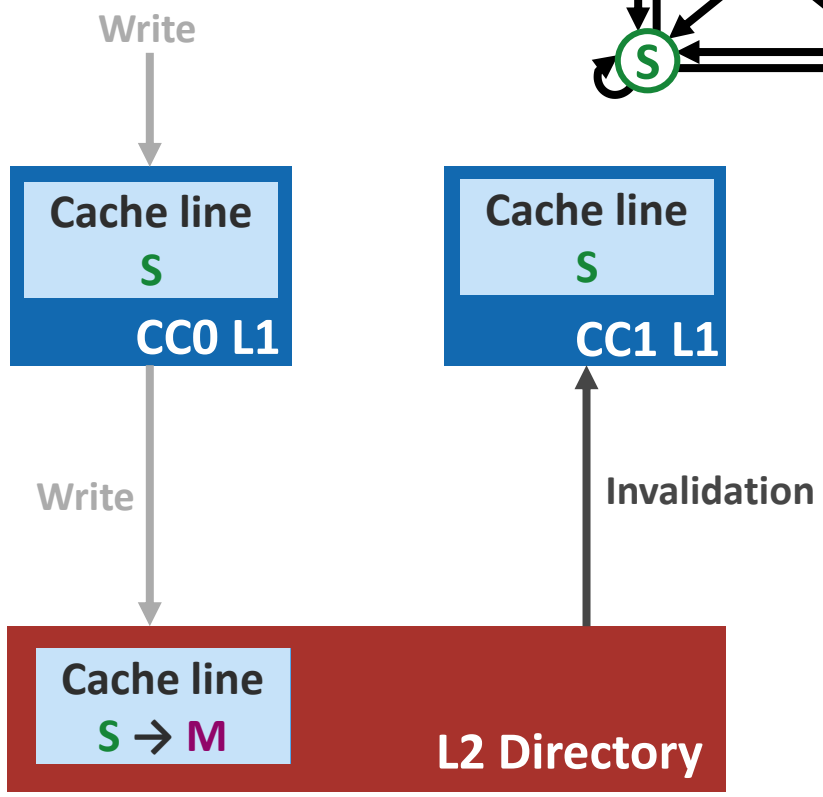


L1 Write Hit on Shared Line

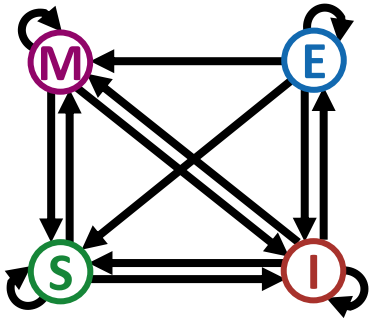


Shared! L1 cannot commit immediately!

1. Directory gets informed
2. **Directory**
   1. Update coherence state and sharer list
   2. **Send Invalidation**

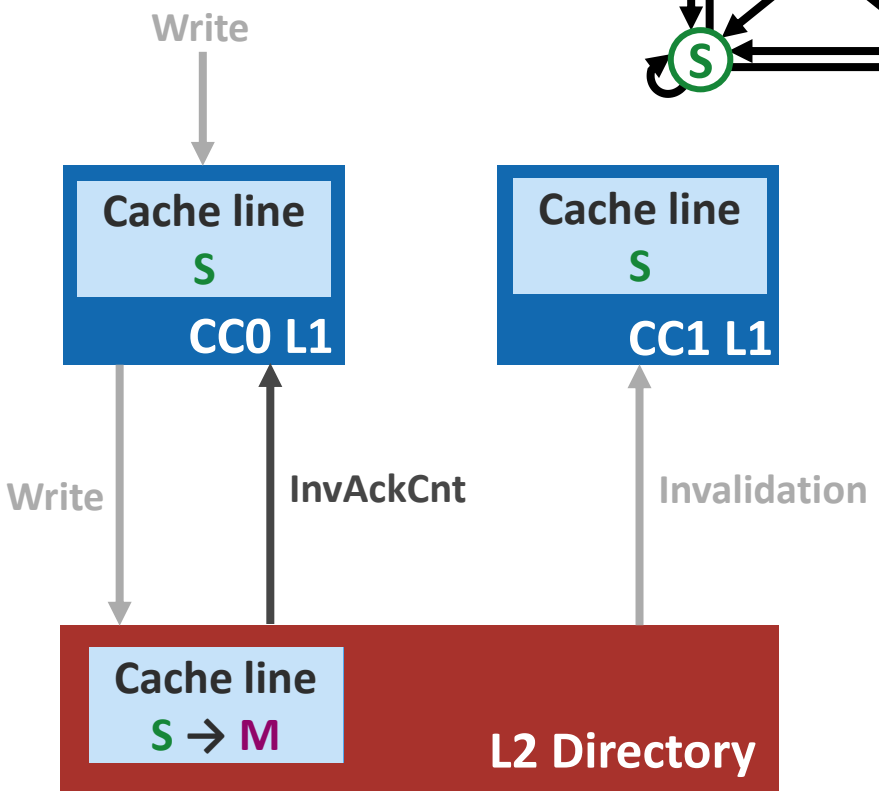


L1 Write Hit on Shared Line

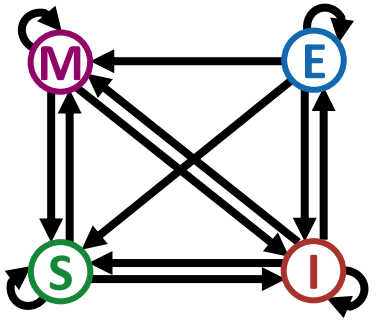


Shared! L1 cannot commit immediately!

1. Directory gets informed
2. **Directory**
   1. Update coherence state and sharer list
   2. Send Invalidation
   3. **Inform the requester: how many Acks to expect?**

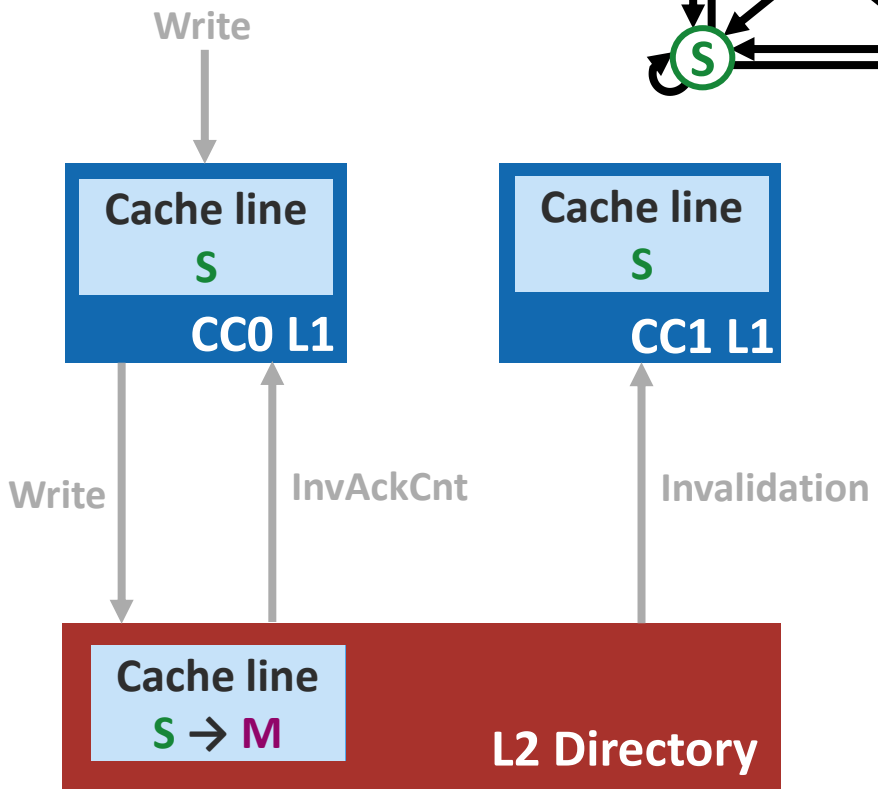


L1 Write Hit on Shared Line

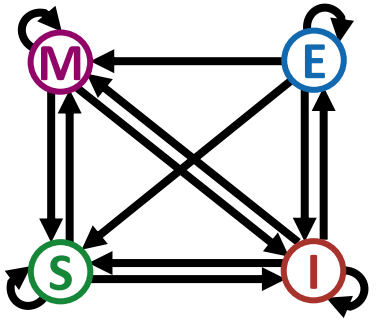


Shared! L1 cannot commit immediately!

1. Directory gets informed
2. Directory
   1. Update coherence state and sharer list
   2. Send Invalidation
   3. Inform the requester: how many Acks to expect?
3. **Other sharers**
   1. **Invalidates their own copies**

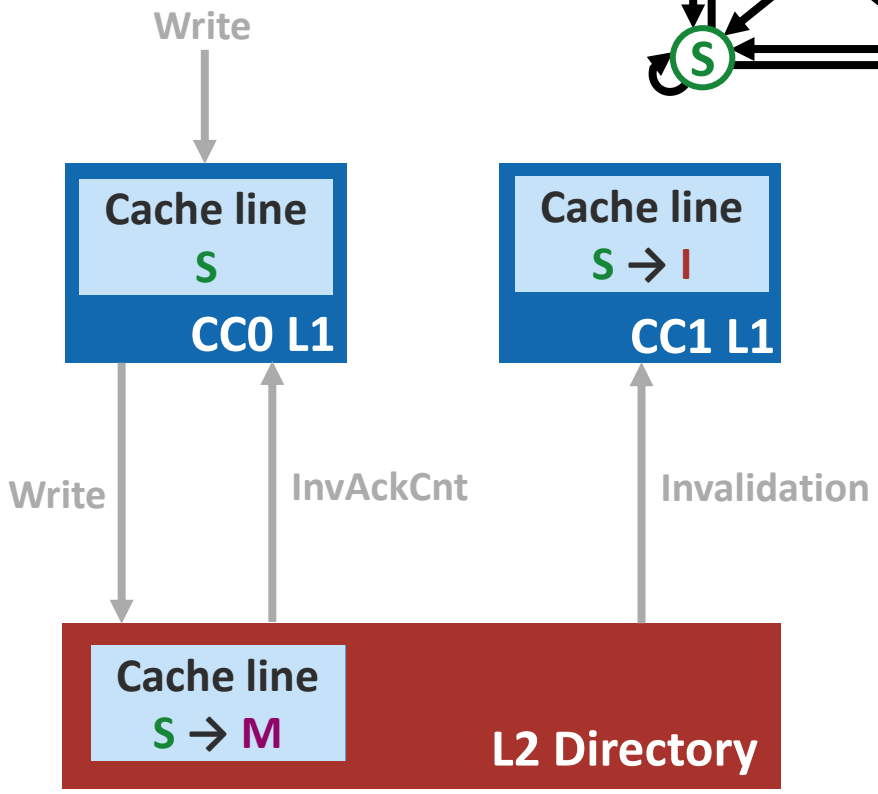


L1 Write Hit on Shared Line

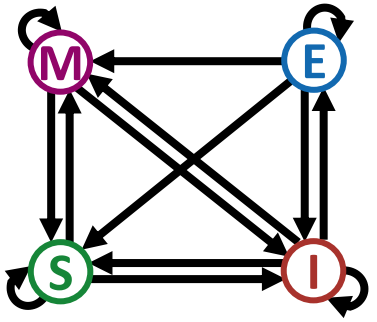


Shared! L1 cannot commit immediately!

1. Directory gets informed
2. Directory
   1. Update coherence state and sharer list
   2. Send Invalidation
   3. Inform the requester: how many Acks to expect?
3. **Other sharers**
   1. **Invalidates their own copies**

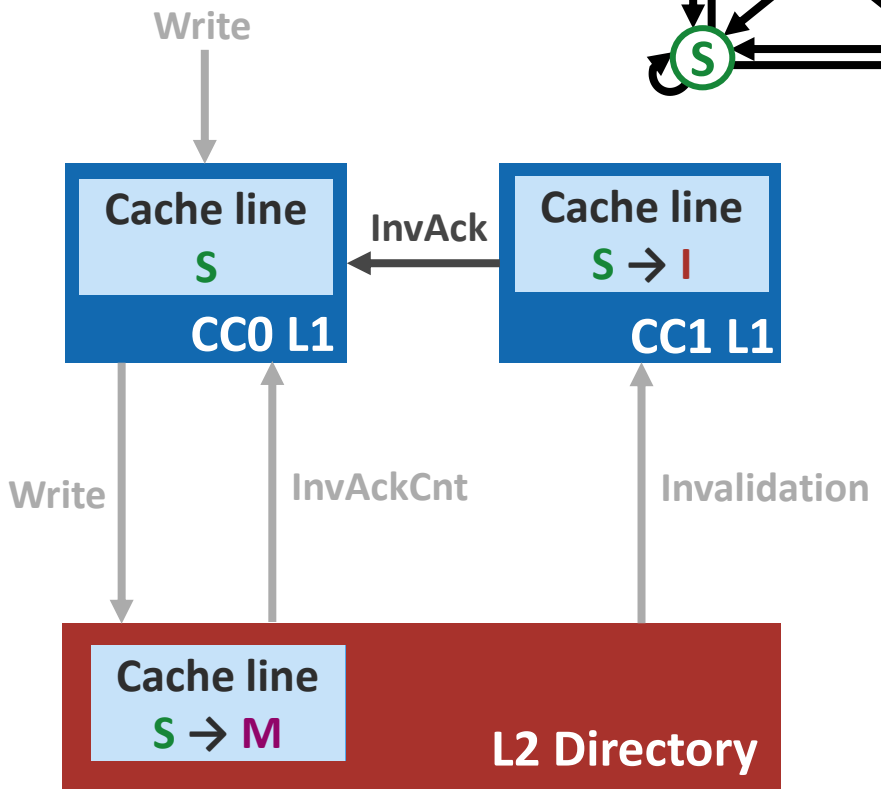


L1 Write Hit on Shared Line

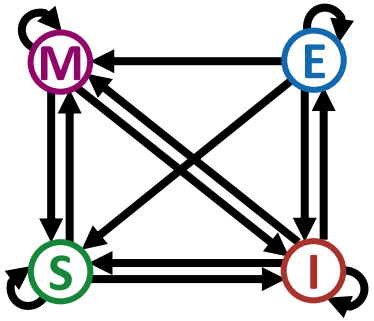


Shared! L1 cannot commit immediately!

1. Directory gets informed
2. Directory
   1. Update coherence state and sharer list
   2. Send Invalidation
   3. Inform the requester: how many Acks to expect?
3. **Other sharers**
   1. Invalidates their own copies
   2. **Send acknowledgements**



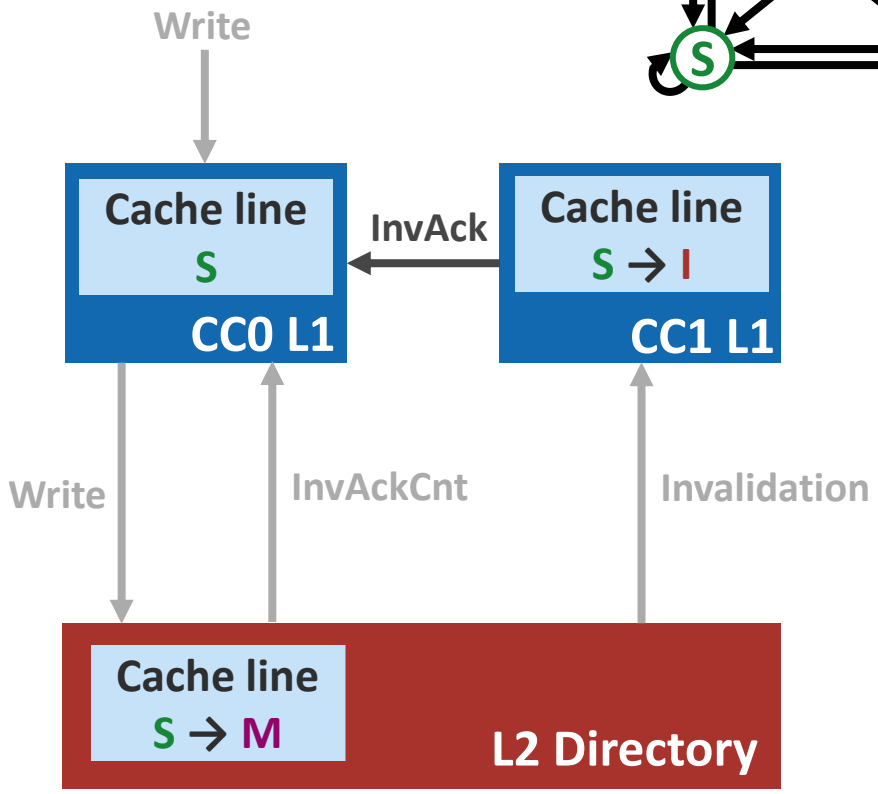
L1 Write Hit on Shared Line



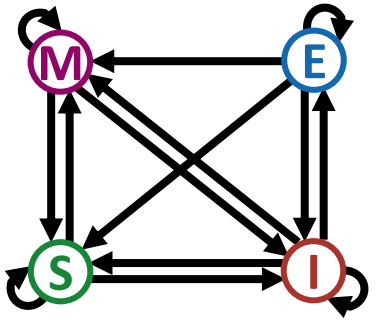
Shared! L1 cannot commit immediately!

1. Directory gets informed
2. Directory
   1. Update coherence state and sharer list
   2. Send Invalidation
   3. Inform the requester: how many Acks to expect?
3. Other sharers
   1. Invalidates their own copies
   2. Send acknowledgements
4. **After receiving all Acks**

   Commit write



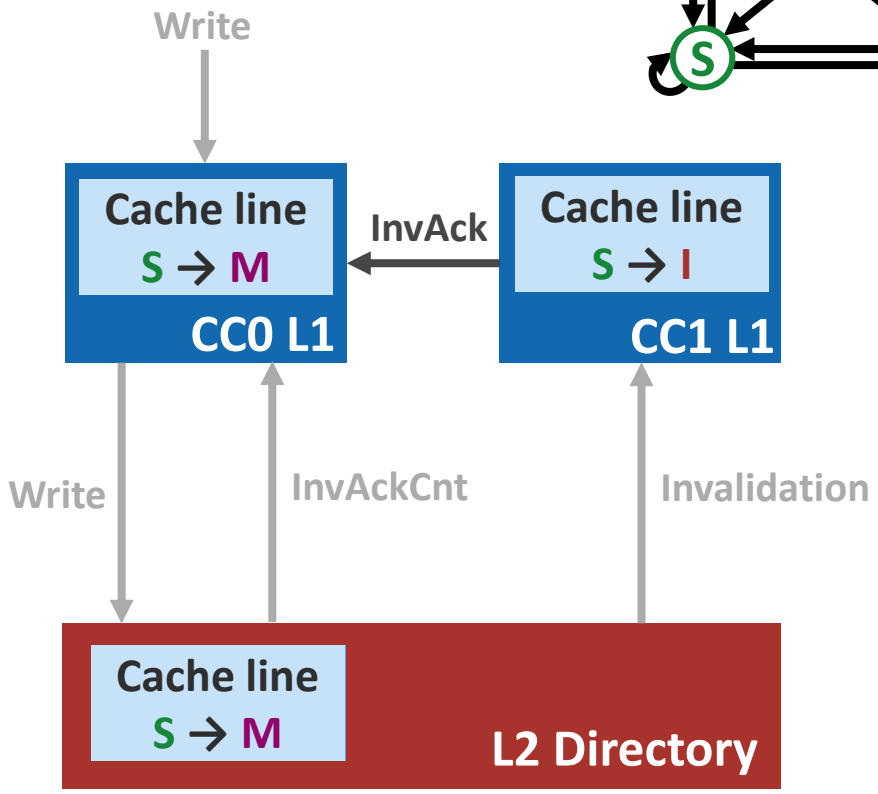
L1 Write Hit on Shared Line



Shared! L1 cannot commit immediately!

1. Directory gets informed
2. Directory
   1. Update coherence state and sharer list
   2. Send Invalidation
   3. Inform the requester: how many Acks to expect?
3. Other sharers
   1. Invalidates their own copies
   2. Send acknowledgements
4. **After receiving all Acks**

   Commit write



Contributions

- **Extend the cache hierarchy of CachePool**
  - Integrated write-through per-core private L1 data cache
- **Developed a light-weight coherence protocol**
  - L2 Tag-embedded coherence directory
  - Lightweight directory controller
  - Directory-based MESI protocol
- **Explored impact on performance and physical design**

Evaluation Setup

Cachepool configuration

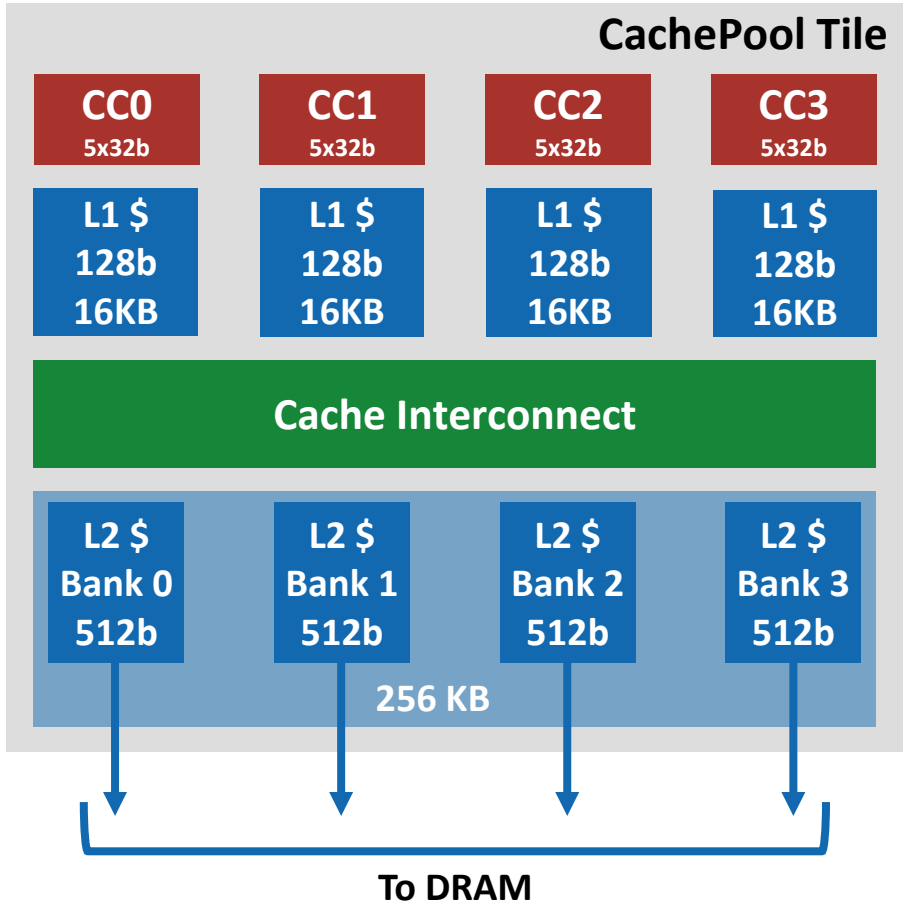
- Single-tile with 4 CCs
- 32b data and address width

L1 configuration

- 4-way associative, 16KB per CC
- 4×128b cache line
- 8-set 4-way associative MSHR
- Write-through with write buffer

L2 configuration

- 4-way associative, 256KB organized in 4 banks
- 512b cache line



Performance Results

Kernels tested on:

Floating point dot product (FDOTP)

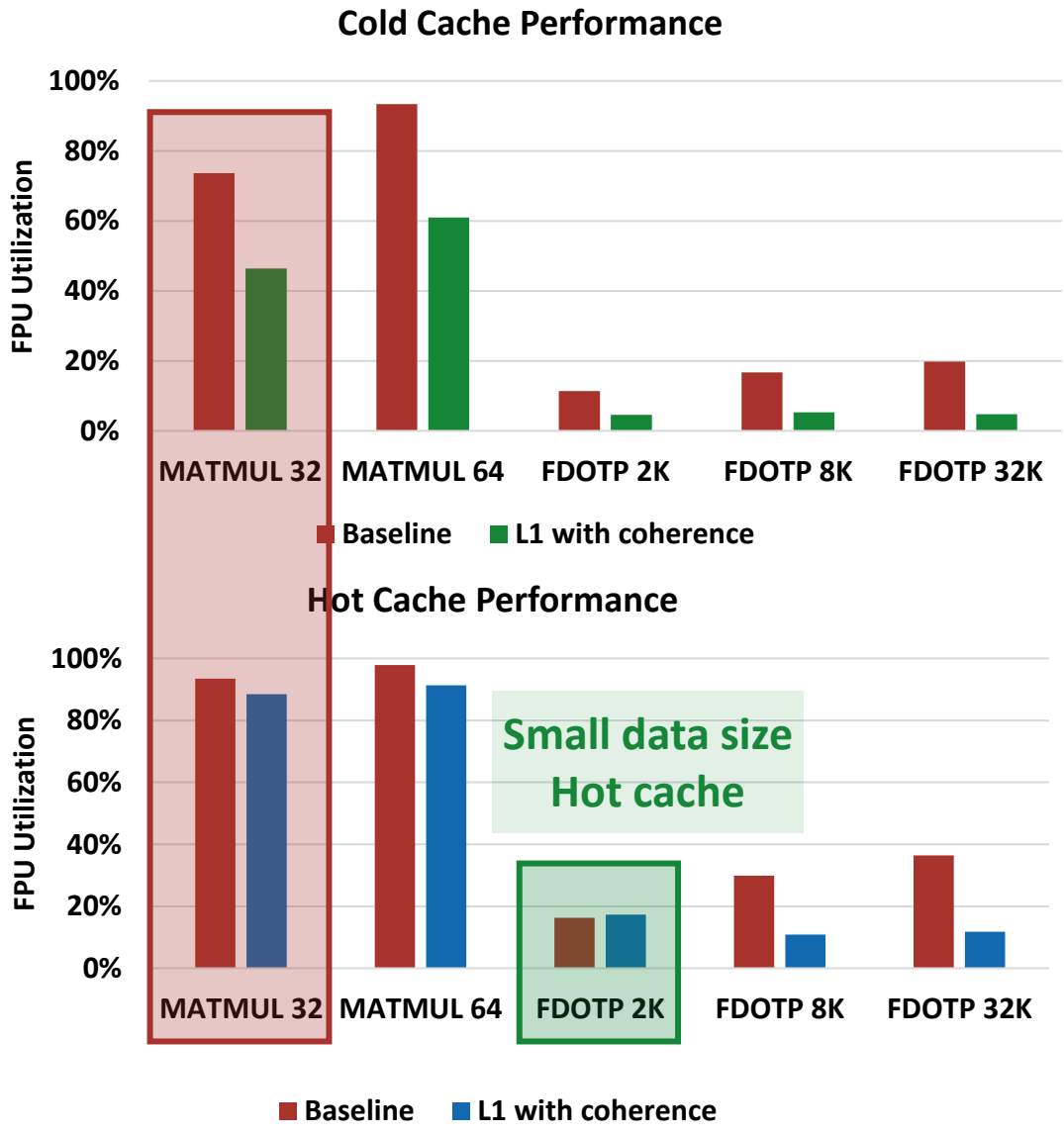
- **2K, 8K and 32K vector size**

FP matrix multiplication (FMATMUL)

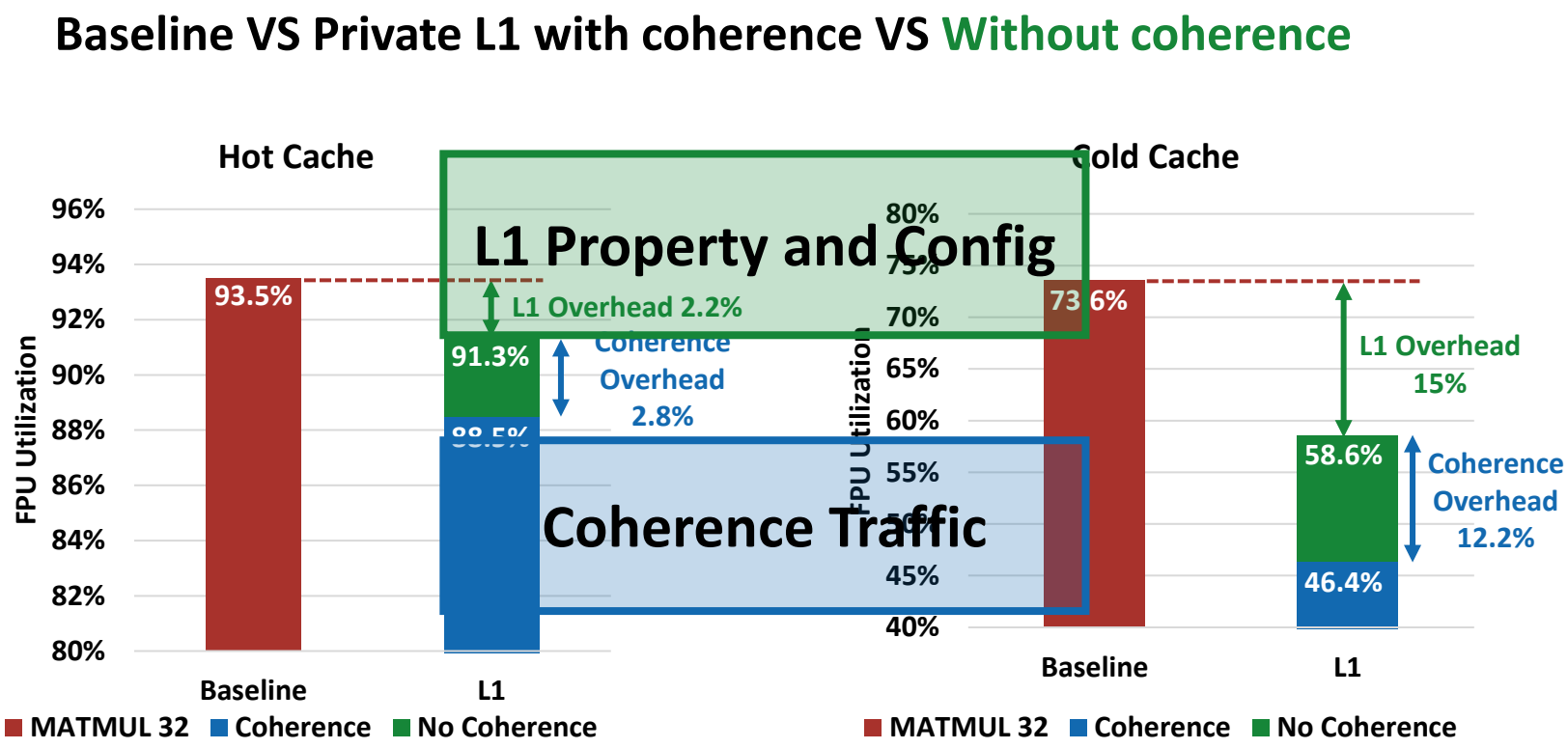
- **M N K = 32 and 64**

FPU Utilization

Performance degradation



Performance Results



Performance Results

L1 Property and Config

One-cycle hit latency kept with coherence

MSHR

Replacement policy

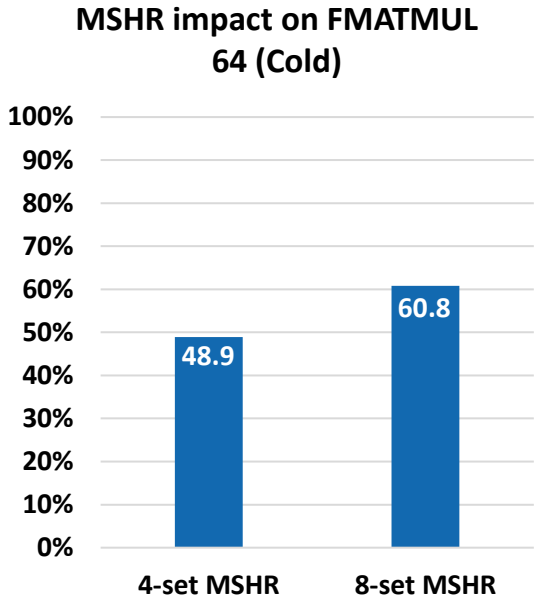
Miss penalty

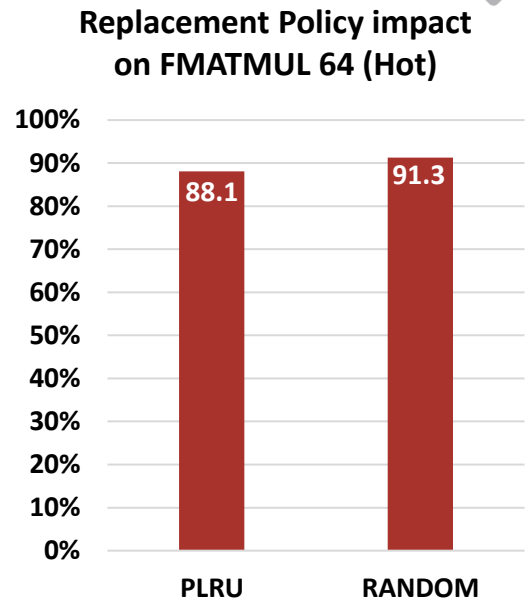
Coherence Traffic

Best-case 2-cycle access

Meta bank traffic congestion

Conservative stall and control flow





L1 Miss Rate	Cold Cache	Hot Cache
FDOTP 2K	99.2%	0%
FDOTP 8K	99.8%	71.7%
FDOTP 32K	99.9%	75.1%
FMATMUL 32	35.2%	2.5%
FMATMUL 64	33.5%	26.9%

Physical Design

Stage: Fully routed

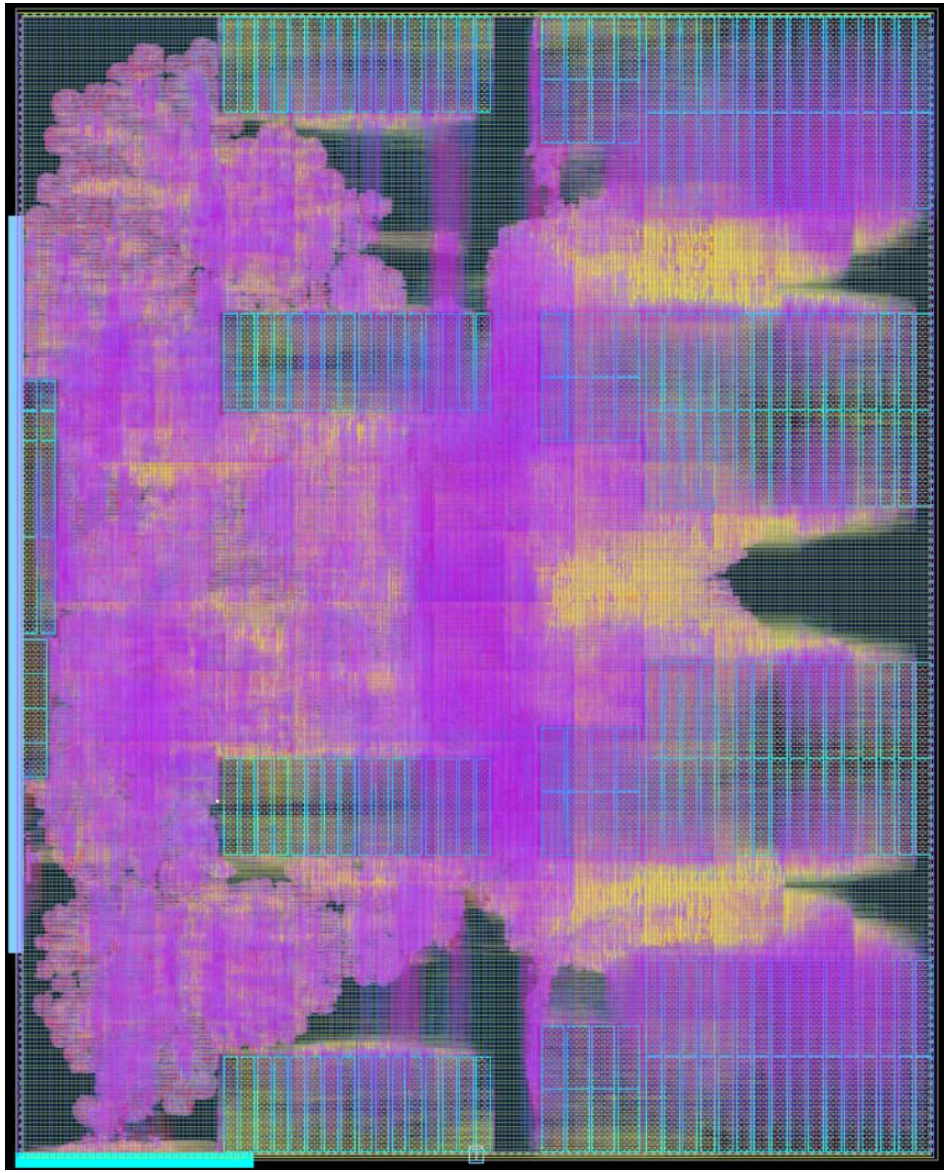
Technology: GF12

Tool: Synospys Fusion Compiler

Clock frequency: 1GHz

Timing

- **Closed at typical corner**
- **Worst corner: 890MHz**
  - Baseline design: 944 MHz
  - 50 MHz degradation compared to the baseline
- **Critical path with slack of -0.119ns**
  - From HPDcache MSHR, via core
  - To HPDcache data SRAM



Physical Design – Logic Area

Baseline (kGE): total 8778

Core	L2 Ctrl	L2 Data	L2 Tag	Others
2.6K	1.1K	3.3K	620	1K
30%	13%	38%	7%	12%

With L1 and Coherence (kGE): total 11416

Core	L2 Ctrl	L2 Data	L2 Tag	Others	L1 Data	L1 Tag
2.6K	806	3.3K	834	1.4K	1.5K	475
24%	7%	30%	7%	12%	14%	4%

Data SRAM

L2 Dir Ctrl
100
0.9%

Tag extension

L1 Ctrl
247
2%

Coh. Xbar
41
0.4%

L1 Coh. Logic
13
0.1%

Physical Design – Logic Area

Baseline (kGE): total 8778

Core	L2 Ctrl	L2 Data	L2 Tag	Others
2.6K	1.1K	3.3K	620	1K
30%	13%	38%	7%	12%

With L1 and Coherence (kGE): total 11416

Core	L2 Ctrl	L2 Data	L2 Tag	Others	L1 Ctrl	L1 Data	L1 Tag
2.6K	806	3.3K	834	1.4K	247	1.5K	475
24%	7%	30%	7%	12%	2%	14%	4%

Minimal Coherence Area Cost

Coherence Xbar	L2 Dir Ctrl	L1 Coh.
41	100	13
0.4%	0.9%	0.1%

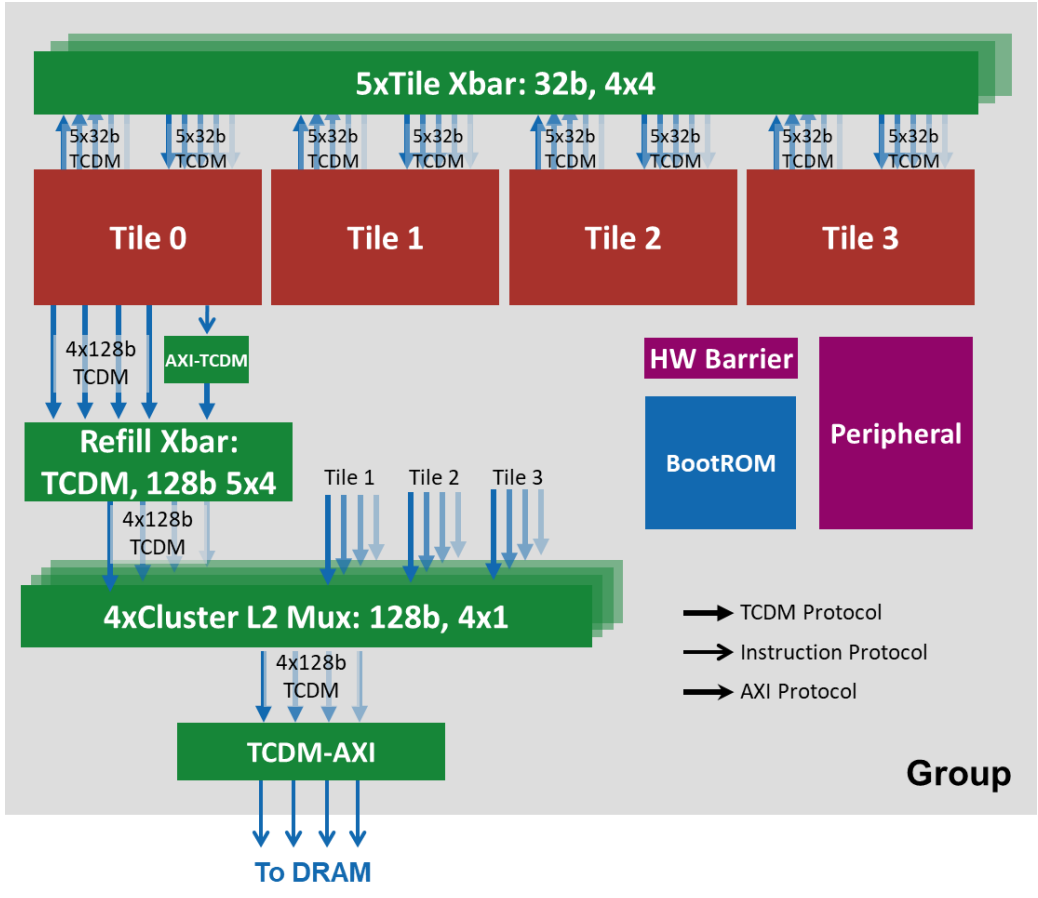
1.5% of total area

Key Achievements and Future Directions

- **L1 private cache** on CachePool
- Light-weight MESI-based **coherence protocol**
- Performance bottleneck exploration
  - L1 cache and coherence traffic
- Physical design
  - Timing closed at typical corner
  - Minimal coherence area cost of **1.5%**

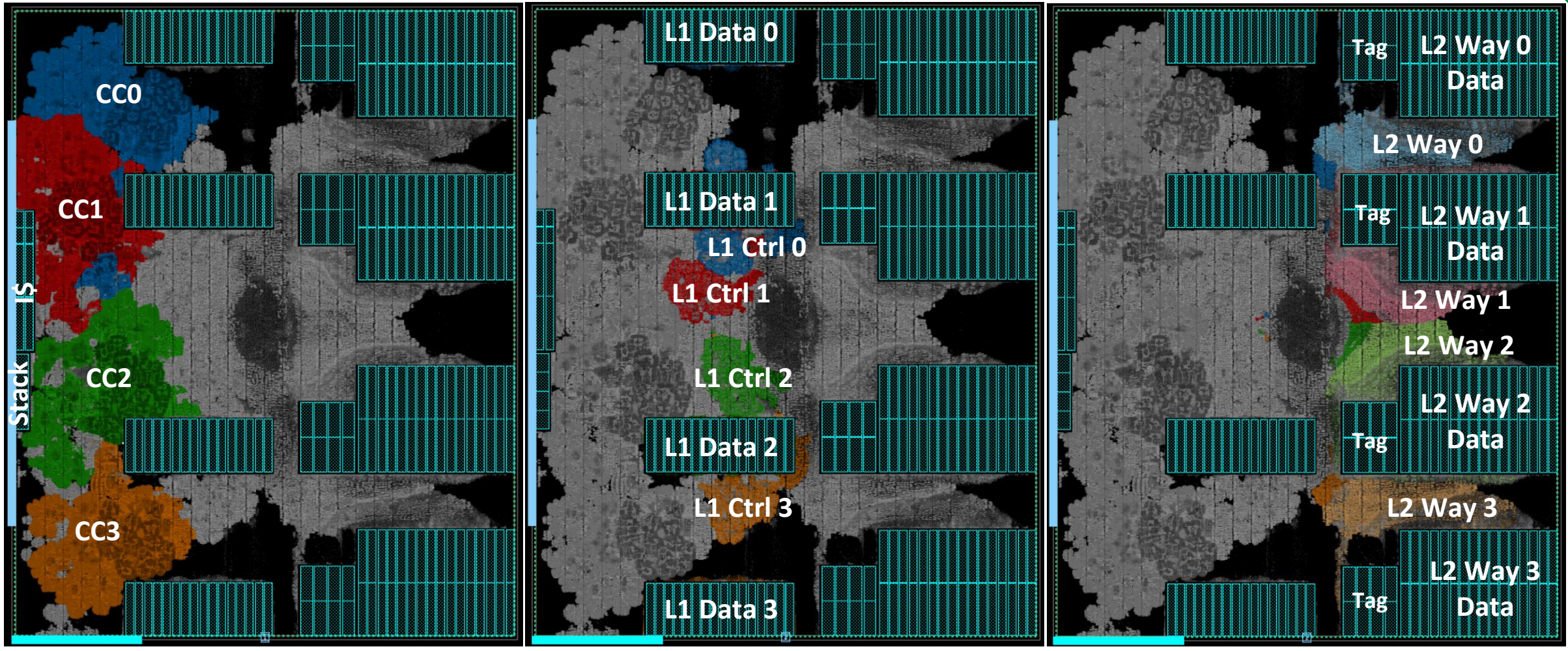
Future Work and Outlooks

- Lazy write-through from L1 to L2
- Multi-tile CachePool support
- Performance optimizations



Thank you!

Backup: Colored Layout



Backup: Critical Path

hpdcache_mshr.check_cache_set_q_reg
(module: **hpdcache_miss_handler / i_hpdcache_mshr**)
↓
local MSHR decision / set-check combinational logic
↓
cachepool_cc / Spatz interface
signal: **fp_lsu_mem_rsp_i[9]**
↓
Spatz FPU sequencer logic
(module: **i_cachepool_cc_i_spatz / gen_fpu_sequencer_i_fpu_sequencer**)
↓
FPU memory request generation
signal: **fp_lsu_mem_req_o[0]**
↓
cache control / ready-handshake logic
(between cachepool_cc and hpdcache control)
↓
hpdcache_ctrl internal control logic
(module: **hpdcache_ctrl**)
↓
hpdcache_memctrl SRAM control generation
signal: **i_hpdcache_memctrl_i_data_cs_10_**
↓
data SRAM request path
signal: **gen_data_sram.../req_i[0]**
↓
data SRAM clock-gate enable
endpoint: **gen_data_sram.../i_ckg_i_clk_gate/E**

Corner: sspg_sigcmax_0p72v_125c
Scenario: func_sspg_sigcmax_0p72v_125c

clock reconvergence pessimism	1.387
library setup time	1.314
data required time	1.314
---	---
data required time	1.314
data arrival time	-1.433
---	---
slack (VIOLATED)	-0.119

Backup: L1 Protocol Table

State	Read	Write	Get	Evict	Replace	Inv	Evict Ack	Exc Data	Data	Inv Ack	Last Inv Ack
I	Send read to Dir IS^D	Send Write to Dir I	-	-	-	-	-	-	-	-	-
IS^D	Stall	Stall	Send GetAck to Dir	Stall	-	Send InvAck to Req $E \rightarrow S$	-	-	-	-	-
S	Hit	Send Write to Dir SM^A	-	Send Evict to Dir SI^A	Send Evict to Dir	Send InvAck to Req I	-	-	-	-	-
SM^A	Hit	Stall	-	Stall	-	Send InvAck to Req I	-	-	-	Ack $\rightarrow M$	-
M	Hit or Refill Req	Hit	Send GetAck to Dir Flush Wbuf S	Send Evict to Dir MI^A	Send Evict to Dir	Send InvAck to Req I	-	-	-	-	-
E	Hit	Hit M	Send GetAck to Dir S	Send Evict to Dir EI^A	Send Evict to Dir	Send InvAck to Req I	-	-	-	-	-
MI^A	Stall	Stall	Send GetAck to Dir Flush Wbuf	Stall	-	-	I	-	-	-	-
EI^A	Stall	Stall	Send GetAck to Dir	Stall	-	-	I	-	-	-	-
SI^A	Stall	Stall	-	Stall	-	Send InvAck to Req II^A	I	-	-	-	-
II^A	Stall	Stall	-	Stall	-	-	I	-	-	-	-

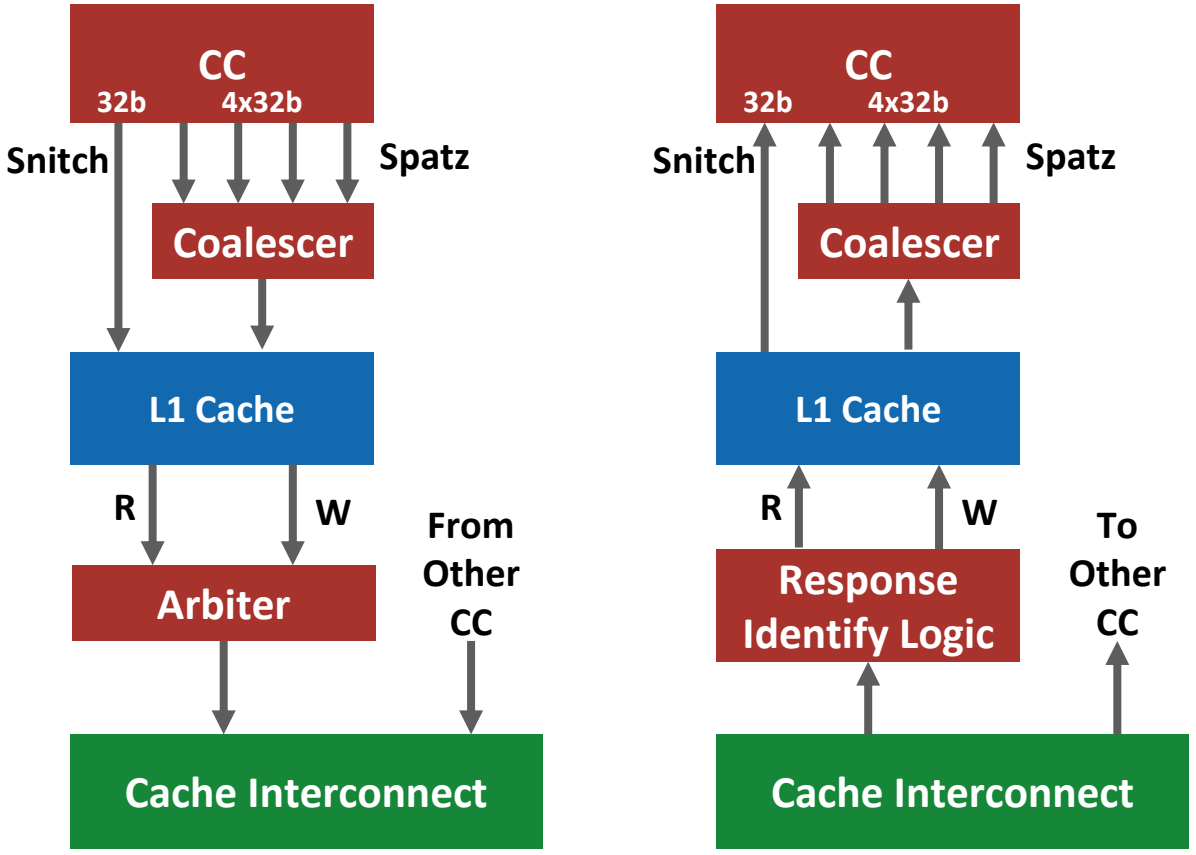
Table 4.2.: Detailed Coherence Protocol Table (L1 Cache Controller).

Backup: Directory Protocol Table

State	**Read**	**Write**	**GetAck**	**Evict S**	**Evict M** (Owner)	**Evict M** (Non-owner)	**Evict E** (Owner)	**Evict E** (Non-owner)
I	Send exclusive data. *E*	Update L2 data. *M*	-	-	-	Send EvictAck .	-	Send EvictAck .
S	Send data Update sharers.	Update L2 data. Send Inv to sharers. *M*	-	Remove Req from sharers. Send EvictAck.	-	Remove Req from sharers. Send EvictAck.	-	Remove Req from sharers. Send EvictAck.
E	Send Get to L1 ES^A	Update L2 data. Send Inv to owner. Update sharers. *M*	-	Send EvictAck.	Write to Memory. Send EvictAck. *I*	Send EvictAck.	Send EvictAck. Clear sharers (owner).	Send EvictAck.
ES^A	Stall	Stall	Send latest data. Update sharers. *S*	-	Send EvictAck. Respond to pending read Req with latest data. *E* (new owner)	Send EvictAck. Respond to pending read Req with latest data. *E* (new owner)	Send EvictAck. Respond to pending read Req with latest data. *E* (new owner)	Send EvictAck. Respond to pending read Req with latest data. *E* (new owner)
M	Send Get to L1 ES^A	Update L2 data. Send Inv to owner. Update sharer.	-	Send EvictAck.	Write to Memory. Send EvictAck. *I*	Send EvictAck.	-	Send EvictAck.

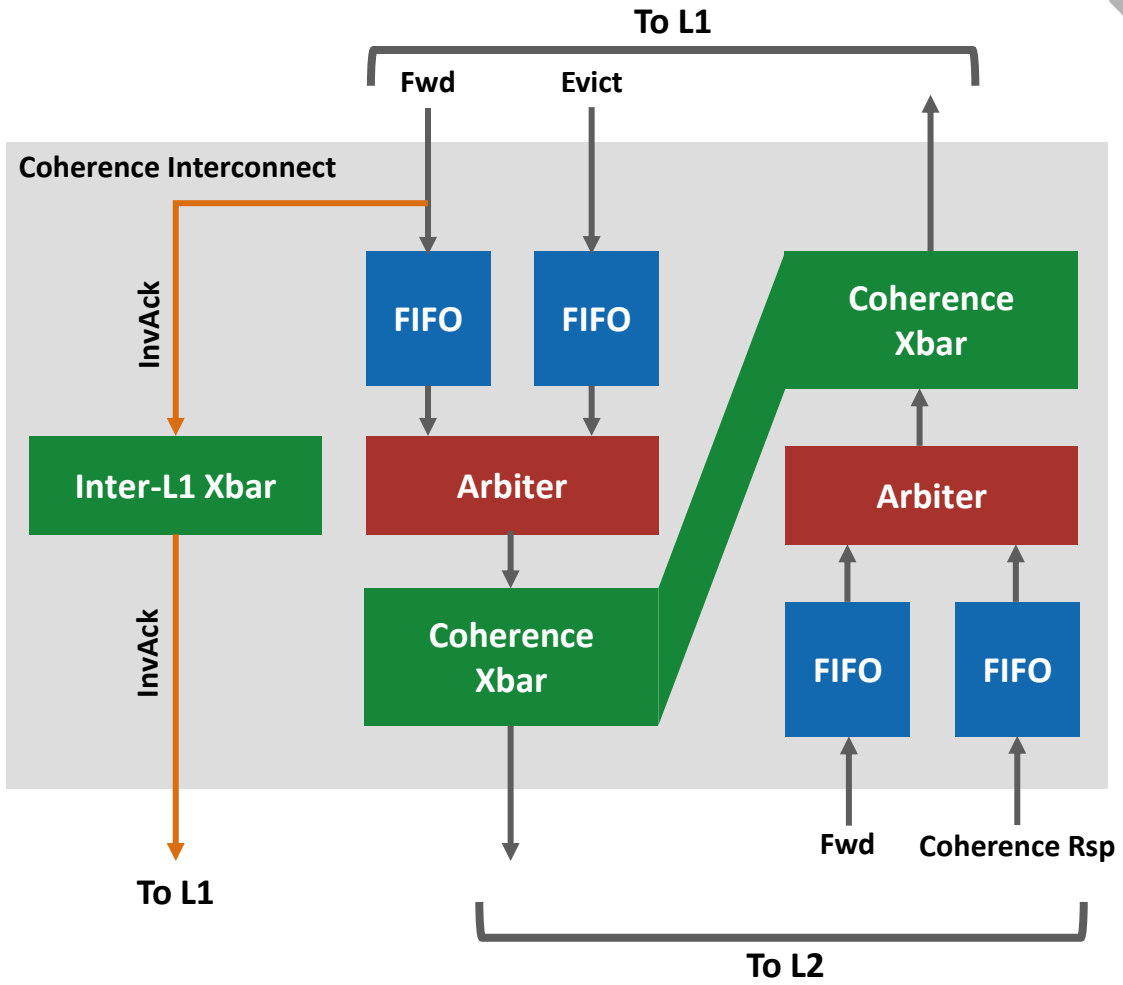
Table 4.3.: Detailed Coherence Protocol Table (L2 Directory Controller).

Backup: Coalescing Datapath



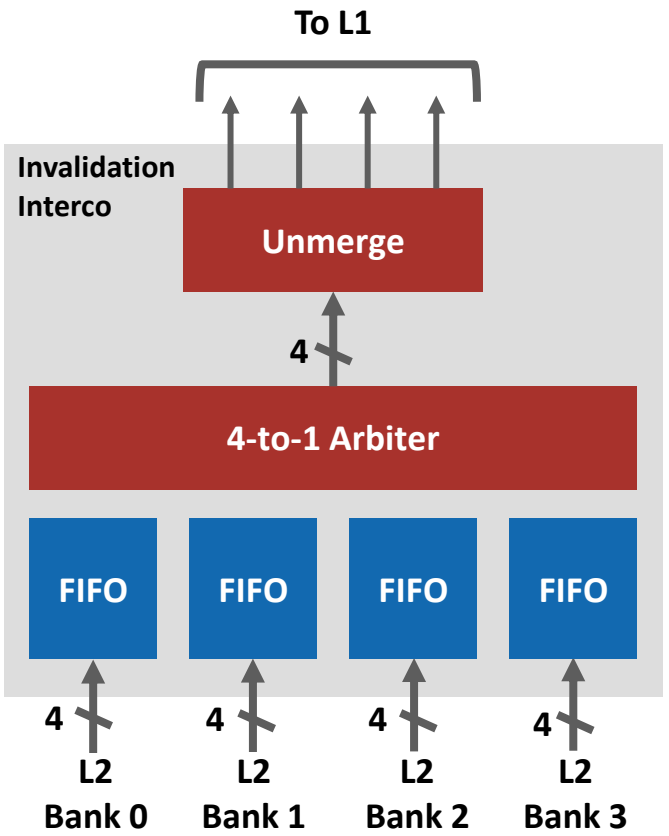
Backup: Coherence Message Network

- Crossbar interconnect to route the messages
- Incoming messages are first buffered using FIFOs
  - To tolerate concurrent requests
- Arbiter select between competing messages
- Chosen message forwarded via the crossbar
  - Same bank interleaving mechanism as the cache interconnect
- Special case 1: inter-L1 messages (InvAck)
  - Separate crossbar used



Backup: Coherence Message Network

- Crossbar interconnect to route the messages
- Incoming messages are first buffered using FIFOs
  - To tolerate concurrent requests
- Arbiter select between competing messages
- Chosen message forwarded via the crossbar
  - Same bank interleaving mechanism as the cache interconnect
- Special case 1: inter-L1 messages (InvAck)
  - Separate crossbar used
- Special case 2: invalidations
  - Worst case 16-to-4 broadcast
  - Special handling and independent interconnect



Backup: Insitu-cache

- Non-blocking cache that tolerates long-latency accesses directly to DRAM
- In-cacheline handling for read and write misses
- Dynamic Subentry Expansion (DSE)
  - Extending miss handling capacity under high-latency conditions.
- Dynamic Subset Least Recent Used (DS-LRU)
  - Reclaims wasted cache lines for miss handling